

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Exploratory Analysis of Segmentation Techniques for Multiphase Flow Experimental Images

Jorge Levi Perdigoto da Costa

WORKING VERSION



Mestrado em Engenharia Informática e Computação

Supervisors: Mónica Silva, Zafeiris Kokkinogenis

January 20, 2026

Exploratory Analysis of Segmentation Techniques for Multiphase Flow Experimental Images

Jorge Levi Perdigoto da Costa

Mestrado em Engenharia Informática e Computação

January 20, 2026

List of Figures

1.1	Conceptual link between multiphase-flow physics, imaging challenges, segmentation requirements, and modern deep-learning solutions.	3
3.1	End-to-end research workflow adopted in this dissertation: dataset preparation and annotation, repeated model experiments (synthetic training, real baseline training, synthetic-to-real transfer learning, and transfer learning with augmentation), evaluation on a held-out experimental test split, and an annotation-efficiency study using model-assisted labeling.	17
3.2	Illustration of the frame-selection step used to build the annotated experimental dataset. .	20
3.3	Representative examples from the 14 selected frames, illustrating variability in bubble density, magnification, and contrast targeted in the annotation-efficiency study.	28
3.4	Manual polygon-based annotation in Label Studio (Mode A), used when bubble contours are irregular or require careful contour.	28
3.5	Manual brush-based annotation in Label Studio (Mode A), used for regular and approximately circular/elliptical bubbles.	29
3.6	Mode B example where synthetic pre-annotations closely match the target masks and require only minor edits.	30
3.7	Mode B correction examples: removal of incorrect instances, and final corrected annotation.	30
3.8	Mode B failure example where pre-annotations are significantly incorrect and require substantial manual correction.	31
3.9	Mode C (SAM2-assisted refinement) in Label Studio: example interaction, resulting mask, and a challenging case requiring additional refinement.	31
4.1	Overview comparison across the 12 experiments (3 backbones $\times$ 4 regimes). All metrics are computed on the same real test split under fixed evaluation thresholds.	39
4.2	Effect of synthetic-to-real transfer learning on the real test split (Real vs. Transfer): mean Dice (union-mask) across backbones.	40
4.3	Effect of synthetic-to-real transfer learning on the real test split (Real vs. Transfer): mean absolute count error $ \Delta N $ across backbones.	41
4.4	Effect of synthetic-to-real transfer learning on the real test split (Real vs. Transfer): mean signed count error ΔN (Pred – GT) across backbones.	41
4.5	TensorBoard validation mAP (AP@[0.50:0.95]) versus epoch for <code>resnet50_fpn</code> : Real vs. Transfer training.	42
4.6	TensorBoard validation mAP (AP@[0.50:0.95]) versus epoch for <code>resnet50_fpn_v2</code> : Real vs. Transfer training.	42

4.7	TensorBoard validation mAP (AP@[0.50:0.95]) versus epoch for <code>resnet101_fpn</code> : Real vs. Transfer training.	43
4.8	Effect of augmentation during transfer learning (Transfer vs. Transfer+Aug): mean Dice (union-mask) across backbones.	44
4.9	Effect of augmentation during transfer learning (Transfer vs. Transfer+Aug): mean absolute count error $ \Delta N $ across backbones.	44
4.10	Effect of augmentation during transfer learning (Transfer vs. Transfer+Aug): mean signed count error ΔN (Pred – GT) across backbones.	45
4.11	Empirical CDF of absolute count error $ \Delta N $ for <code>resnet50_fpn</code> (Transfer vs. Transfer+Aug).	46
4.12	Empirical CDF of absolute count error $ \Delta N $ for <code>resnet50_fpn_v2</code> (Transfer vs. Transfer+Aug).	46
4.13	Empirical CDF of absolute count error $ \Delta N $ for <code>resnet101_fpn</code> (Transfer vs. Transfer+Aug).	47
4.14	Inference throughput (FPS) by backbone in the transfer regime on the real test split (same hardware and evaluation pipeline).	48
4.15	Mean Dice (union-mask) by backbone in the transfer regime on the real test split.	49
4.16	Speed–accuracy trade-off in the transfer regime (Dice versus FPS).	49
4.17	Stratified Dice (union-mask) in the transfer regime by bubble density (GT_NumBubbles bins).	50
4.18	Stratified mean absolute count error $ \Delta N $ in the transfer regime by bubble density (GT_NumBubbles bins).	51
4.19	Stratified mean signed count error ΔN (Pred–GT) in the transfer regime by bubble density (GT_NumBubbles bins).	51
4.20	Stratified Dice (union-mask) in the transfer regime by bubble scale (GT_MeanArea bins).	52
4.21	Stratified mean absolute count error $ \Delta N $ in the transfer regime by bubble scale (GT_MeanArea bins).	53
4.22	Qualitative comparison for ImageIdx=12. Top: <code>resnet50_fpn_v2</code> real-only. Middle: <code>resnet50_fpn_v2</code> transfer. Bottom: <code>resnet101_fpn</code> transfer, showing overcounting despite visually plausible overlap.	54
4.23	False-positive example for ImageIdx=5. Both models are challenged by bubble-like structures that are not true bubbles, leading to spurious instances and increased counting error.	55
4.24	Dense, low-visibility case for ImageIdx=20. Comparison between <code>resnet50_fpn_v2</code> transfer and <code>resnet101_fpn</code> transfer highlights differences in overprediction tendency under darker and denser conditions.	56
4.25	Robust success example for ImageIdx=3 with small/irregular bubbles. The transfer regime remains stable, indicating that improved generalization is not limited to near-circular instances.	57
4.26	Annotation time per image across the three annotation modes. Each coloured line corresponds to one of the 14 selected images.	58
4.27	Boxplot of seconds per bubble for each annotation mode. The combined synthetic pre-annotation + SAM2 workflow (Mode C) reduces annotation cost by nearly 30× compared with manual annotation.	60
4.28	Example of SAM2-assisted refinement applied to synthetic pre-annotations, producing accurate bubble masks along the channel.	61

List of Tables

2.1	Quantitative comparison of representative segmentation approaches for bubbly-flow imagery (NR: not reported explicitly in the cited source).	10
2.2	Representative datasets and dataset-building strategies used in bubble segmentation studies (NR: not reported explicitly in the cited source).	13
3.1	Datasets used in this dissertation and their role in the experimental workflow.	21
3.2	Default training and evaluation parameters used across Mask R-CNN experiments. . . .	25
4.1	Main comparison of the 12 experiments on the real experimental test split (25 images). All runs use the same post-processing thresholds (score_thresh=0.5, mask_thresh=0.5, min_area=50). Metrics are averaged per image. $ \Delta N $ is the mean absolute bubble-count error and ΔN is the signed mean count error (Pred-GT). Inference timing (FPS) is measured on an NVIDIA RTX 2060 Max-Q 6GB (Intel Core i5-10300H host). No empty frames occur in this test split (#EmptyGT=#EmptyPred=0 in all runs).	38
4.2	Real→Transfer deltas on the real test split (Transfer – Real). Positive deltas in Dice/IoU indicate improved overlap; negative deltas in $ \Delta N $ indicate improved counting stability. .	40
4.3	Best validation segmentation mAP (AP@[0.50:0.95]) recorded during training (from the per-epoch metrics CSV exported by the training pipeline).	44
4.4	Transfer→Transfer+Aug changes on the real test split (Transfer+Aug – Transfer). Negative deltas in Dice/IoU indicate reduced overlap; positive deltas in $ \Delta N $ indicate worse counting stability.	45
4.5	Backbone capacity versus inference throughput in the transfer regime (real test).	47
4.6	Stratified performance in the transfer regime by bubble density (bins defined by GT_NumBubbles quartiles on the 25-image real test split).	50
4.7	Stratified performance in the transfer regime by mean bubble size (bins defined by GT_MeanArea quartiles on the 25-image real test split).	52
4.8	Mean, median, and standard deviation of annotation time per image for each mode. . . .	57
4.9	Seconds per bubble across different annotation modes.	59

List of Abbreviations and Symbols

AI	Artificial Intelligence
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset)
CSPDarknet53	Cross Stage Partial Darknet-53 (YOLO feature extractor)
FPN	Feature Pyramid Network
FPS	Frames Per Second
GAN	Generative Adversarial Network
HSV	High-Speed Video
IoU	Intersection over Union
MLP	Multilayer Perceptron
mAP	Mean Average Precision
MOTA	Multiple Object Tracking Accuracy
NSD	Nucleation Site Density
R-CNN	Region-based Convolutional Neural Network
RDC	Radial Distance Correction
RoI	Region of Interest
SAM	Segment Anything Model
SNN	Siamese Neural Network
U-Net	A deep learning segmentation model
YOLO	You Only Look Once (Object Detection Model)

Chapter 1

Introduction

1.1 Context and Motivation

Multiphase flow, in which gas bubbles are dispersed in a continuous liquid phase [9], is fundamental to many industrial and natural processes. These systems are characterized by gas bubbles that interact with each other and the surrounding liquid, often coalescing, deforming, and breaking up under turbulent conditions.

The complexity of multiphase flow lies in its dynamic nature, where the size, distribution, and morphology of the bubble directly affect critical processes such as mass transfer and chemical reaction rates [34]. In particular, the morphology of bubbles, their size, shape, and arrangement, is of the most importance to determine the efficiency of processes such as gas-liquid mass transfer and heat transfer [32]. Other important parameters are the gas fraction distribution, the velocity of the bubbles, and eventually their trajectory through the prevailing flow condition [14]. Precise measurement of bubble size and shape is essential for understanding the factors that influence bubble morphology. These measurements allow for a better understanding of how different bubble shapes form, which in turn makes it possible to control bubble morphology [32].

In industrial reactors such as stirred tanks and bubble columns, bubble size and morphology strongly influence gas-liquid mass transfer and overall process efficiency, motivating accurate characterization of bubble populations [32, 9].

Segmentation in image processing, the process of partitioning an image into meaningful regions, helps to analyze multiphase flows by extracting quantitative information and gaining a deeper understanding of flow dynamics. By segmenting individual bubbles, it is not only possible to obtain bubble morphology, but also to track their spatial and temporal evolution over time [28]. This enables the measurement of important flow characteristics like bubble trajectories, growth rates, and velocity fields, using metrics such as nucleation site density (NSD), bubble departure frequency, bubble lifetime, and waiting times between departures [19].

Traditional methods of bubble segmentation, such as the Hough transform, the breakpoint method, and the watershed method, are highly sensitive to specific sets of bubble images [28]. These methods often fail when confronted with varying lighting conditions, complex bubble shapes, reflections, shadows, and overlapping bubble clusters. Other conventional segmentation methods, such as edge detection, threshold segmentation, and the snake model, are effective under certain conditions but suffer from limited performance and accuracy when dealing with complex images [32]. Additionally, manual identification and analysis of bubble boundaries is labor-intensive, time-consuming, and prone to human error, especially in large datasets.

In contrast, deep learning methods, like convolutional neural networks (CNNs), offer more flexibility and scalability in addressing these challenges [28]. Advanced segmentation techniques, particularly those using deep learning, improve accuracy under complex conditions such as poor lighting, shadows, and noisy images [28], which reduces the likelihood of errors in recognizing bubbles, even in noisy or distorted environments, leading to more reliable data for further analysis. Deep learning methods also provide the ability to accurately segment bubbles even when they overlap [9], something traditional methods struggle with.

Improving segmentation in multiphase flow images is not only a technical necessity but also a step towards optimizing industrial processes. Recent advances in machine learning and computer vision, particularly deep learning, offer promising solutions for overcoming the limitations of manual methods. Convolutional neural networks, and more specifically Mask R-CNN models [9], have enabled pixel-level instance segmentation under challenging overlap. Foundation segmentation models such as the Segment Anything Model (SAM) [32] introduce additional flexibility by supporting prompt-based segmentation and efficient refinement, which can be used in model-assisted annotation workflows.

Figure 1.1 summarizes the link between multiphase-flow physics, the imaging artefacts that make segmentation difficult, and the deep-learning approaches that are commonly used to address these challenges. This conceptual framing motivates the problem definition and the methodological choices adopted in this dissertation.

The combination of strong measurement relevance and difficult imaging conditions leads to a practical tension: bubble segmentation must be accurate enough to support reliable physical statistics, yet scalable enough to be applied across experiments and conditions. This tension motivates the problem addressed in the next section, which focuses on limitations of data availability, generalization across setups, and the cost of producing high-quality instance masks.

1.2 Problem

While deep learning methods, such as convolutional neural networks, have demonstrated considerable promise in improving segmentation accuracy, several key challenges hinder their application to multiphase flow analysis. A major limitation lies in the reliance on large amounts of labeled data, which are difficult and costly to obtain for multiphase flows due to their complex and dynamic nature [32]. These

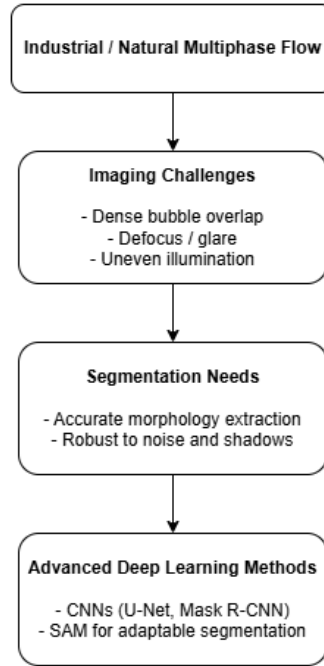


Figure 1.1: Conceptual link between multiphase-flow physics, imaging challenges, segmentation requirements, and modern deep-learning solutions.

models typically demand retraining or careful adaptation to new experimental setups, which increases computational cost and reduces scalability across conditions [32].

Deep learning for bubble segmentation is an active research area, but results can still be strongly dependent on the imaging setup, flow regime, and annotation protocol. This means that even when strong performance is achieved in a given scenario, it may not transfer reliably to new experimental conditions without additional data and adaptation [32, 13].

Existing models, such as the bubSAM framework [32], encounter limitations under certain conditions. For example:

- **Defocused bubbles:** At higher aeration rates, defocused bubbles frequently appear in the images. These artifacts are difficult to remove and are often mistakenly segmented as real bubbles, reducing precision.
- **Small-bubble detection:** The model struggles to capture sufficient detail to accurately segment small bubbles, resulting in reduced recall.
- **Efficiency constraints:** The detection process can be computationally intensive, and practical deployment depends on balancing segmentation quality with processing throughput under a fixed hardware setup.

Therefore, current deep-learning approaches either achieve high accuracy under narrow experimental conditions or generalize poorly when image characteristics vary. This highlights the need for segmentation models and workflows that can adapt across domains, maintain robustness under noise and overlap, and reduce dependence on extensive manual labeling [32, 9, 13].

Another challenge concerns the subjectivity introduced during data labeling and model tuning. Manual annotation and hyperparameter selection can bias the model toward specific conditions, reducing generalization. Integrating pre-trained models and model-assisted labeling tools can mitigate this issue by reducing repetitive manual effort and improving reproducibility of dataset creation [32, 14].

In summary, the core problem lies not in the absence of capable segmentation algorithms, but in their limited adaptability across experimental conditions and their dependence on large annotated datasets. In multiphase flow imaging, variations in bubble morphology, overlap, illumination, and noise can degrade performance when models are applied outside the conditions they were trained on, making generalization and data efficiency central challenges [32, 9, 13].

To address these challenges, this dissertation evaluates deep learning instance segmentation models for experimental multiphase flow images and investigates strategies to reduce reliance on manual annotation. In particular, it studies training on synthetic data followed by fine tuning on real images, and it quantifies the benefits of model assisted annotation workflows that combine synthetic pre annotations with interactive refinement using foundation segmentation models such as SAM and SAM2 integrated in Label Studio [16, 23, 4].

1.3 Research questions

Given the problems identified above, the following research questions were established:

1. How do the selected deep learning instance segmentation models compare when trained and evaluated on the real multiphase flow experimental dataset?
2. What is the impact of training on synthetic bubble data and subsequent fine-tuning on real images on segmentation performance, particularly when real annotations are limited?
3. How much can model-assisted annotation, using synthetic pre-annotations and SAM2-assisted interactive refinement in Label Studio, reduce annotation effort while preserving high-quality bubble masks?

Chapter 2

State of the Art

This chapter reviews the state of the art in bubble identification and segmentation for multiphase-flow imaging, with emphasis on techniques that support reliable quantitative analysis of experimental data. Accurate segmentation masks underpin downstream measurements that are routinely extracted from visualization experiments, including bubble counts, size distributions, and shape-related descriptors [9, 28].

Recent literature shows that segmentation performance is strongly influenced by both the availability of high-quality annotations and the degree of variability between experimental conditions. Instance-level, pixel-accurate masks are time-consuming to produce in dense multiphase flows, which limits the volume and diversity of labeled training data that can be collected [9]. Moreover, models developed for a specific setup often exhibit reduced accuracy when transferred to different operating regimes, optics, or imaging conditions, motivating strategies that improve robustness and cross-domain generalization [9, 15]. As a result, many works explore training paradigms that reduce dependence on extensive manual labeling, including synthetic data generation, transfer learning, domain adaptation, and model-assisted annotation workflows [15, 31, 16, 23, 4].

The remainder of the chapter is organized as follows. Section 2.1 introduces segmentation in multiphase-flow imaging and clarifies the role of semantic and instance formulations. Section 2.2 surveys representative segmentation approaches, from classical baselines to modern deep learning and foundation-model methods. Section 2.3 discusses training strategies under limited annotation budgets, including synthetic-to-real fine-tuning and interactive annotation pipelines. Section 2.4 summarizes available datasets and evaluation practices. Finally, Section 2.5 synthesizes research gaps that motivate the methodological choices explored in this dissertation.

2.1 Segmentation in multiphase flow imaging

In multiphase-flow experiments, segmentation is not an end in itself: it is the interface between raw imagery and quantitative bubble statistics. By converting image content into explicit bubble regions

(and, when possible, per-bubble instances), segmentation enables repeatable measurement of bubble populations across operating conditions and recording setups [9, 28]. This measurement-centric view is important for interpreting segmentation quality: even visually minor boundary errors can bias downstream statistics such as bubble count and size distributions, particularly in dense regimes [9, 13].

2.1.1 Measurement goals supported by segmentation

Segmentation masks support a range of geometric and statistical descriptors that are central to bubbly-flow analysis. At the single-frame level, masks can be used to compute bubble count and size-related quantities (e.g., equivalent diameter/area), as well as shape descriptors that summarize deformation [9]. These measurements are often aggregated into distributions (e.g., size histograms) that characterize the state of the dispersed phase and are used to compare operating regimes or to validate physical models [9].

Beyond static morphology, consistent segmentation across frames enables time-resolved analysis. When masks can be associated over time (explicitly via tracking or implicitly via consistent instance delineation), they support estimation of bubble motion and derived dynamic indicators [28]. In boiling and other transient scenarios, such time-resolved segmentation can be used to study bubble evolution patterns and relate them to heat-transfer-relevant behavior [19]. Recent work also highlights that robust bubble detection/segmentation can be a stepping stone toward more advanced tracking pipelines, including tracking under strong overlap and deformation [34, 14].

2.1.2 Imaging challenges and their implications for measurement reliability

The main difficulty in multiphase imagery is not simply that images are “hard to segment”, but that common artefacts directly translate into biased measurements if the segmentation output is not reliable. Two effects are especially consequential.

First, dense bubbly flows produce frequent apparent overlap because 2D recordings are projections of a 3D scene, increasing occlusion and ambiguity at interfaces [13]. If touching bubbles are merged, bubble counts and size distributions can be systematically distorted; if they are over-split, the opposite bias occurs [9, 13]. This is why many studies emphasize overlap-handling as a core requirement when segmentation outputs are used for bubble-by-bubble statistics [9].

Second, boundary uncertainty (e.g., due to uneven illumination, reflections, blur/defocus, and contrast variability) affects whether masks can be trusted for fine-grained morphology [9, 32]. In practice, these issues concentrate errors around edges and thin interfaces, precisely where small inaccuracies can cause large changes in computed areas for small bubbles, and can propagate into derived statistics in a regime-dependent way [32, 9]. For this reason, segmentation approaches in the literature are commonly evaluated not only by visual plausibility, but by how robustly they preserve measurement-relevant structure under challenging conditions [13, 9].

2.1.3 Semantic vs. instance segmentation

Most approaches can be grouped by the type of output they produce, which determines which measurements they support most directly [13].

Semantic segmentation assigns each pixel a class (e.g., bubble vs. background). This can be sufficient when the objective is phase separation or bulk foreground estimation, but it does not inherently separate touching bubbles, limiting direct per-bubble statistics in dense scenes [13]. In principle, semantic masks can be post-processed to split connected regions (e.g., marker-based separation), but such steps are sensitive to noise and interface ambiguity and can introduce additional failure modes [33, 13].

Instance segmentation outputs one mask per bubble instance, making bubble-level measurements (counting and per-bubble geometry) available by construction, even under partial overlap [9]. This is a key reason why Mask R-CNN-style methods are frequently adopted as baselines for dense bubbly flows, and why instance-level supervision is emphasized when the measurement goal is bubble-by-bubble characterization [9, 13].

Panoptic segmentation combines semantic and instance outputs into a unified representation, but bubbly-flow studies typically focus on semantic or instance formulations depending on whether bulk phase separation or per-bubble statistics is the primary goal [13].

Implication for this dissertation. Because the experimental objectives in this dissertation require bubble-level statistics and support an annotation workflow that benefits from per-instance masks, the remainder of this chapter emphasizes instance segmentation methods and the training strategies commonly used to make them work under limited real annotations [9, 13].

2.2 Segmentation methods

Segmentation approaches for multiphase-flow images can be grouped by the type of representation they produce and by how much of the feature extraction is learned from data. In the literature, this typically spans (i) classical image-processing pipelines, (ii) deep learning models for semantic segmentation (often U-Net-based), (iii) deep learning instance-segmentation models (commonly Mask R-CNN variants) that explicitly separate bubbles under overlap, and (iv) foundation segmentation models that enable promptable and interactive mask generation [13, 9, 16, 23].

2.2.1 Classical approaches

Classical pipelines based on thresholding, edge detection, and marker-based separation can be effective when bubbles are well separated and contrast is stable, but they degrade rapidly with overlap, weak boundaries, or illumination artefacts that are frequent in experimental multiphase imagery [13, 28]. Watershed-based strategies can separate touching regions yet are sensitive to noise and marker errors, which may lead to over-segmentation in cluttered scenes [33]. Similarly, edge-based methods such as the

Canny detector can fail when boundaries are blurred or ambiguous, which is common under reflections, defocus, and strong overlap [8, 13]. These limitations motivate learning-based methods that can adapt feature extraction to the visual variability of experimental data [13].

2.2.2 Deep learning: U-Net and semantic approaches

Semantic segmentation networks aim to classify each pixel (e.g., bubble vs. background), providing a direct estimate of the gas-phase region [13]. U-Net-style architectures are widely adopted for this purpose because their encoder–decoder structure combines contextual information with spatial detail, which supports accurate boundary prediction in many imaging scenarios. In bubble-imaging studies, transfer learning with U-Net variants has been reported to achieve strong performance with relatively limited labeled data, particularly in high-speed visualization settings [26].

However, when the measurement goal requires bubble-by-bubble statistics (Section 2.1), pure semantic masks have an important limitation: touching or overlapping bubbles are often merged into a single connected component, and therefore require additional separation steps (e.g., marker-based splitting or contour post-processing) [13]. In dense regimes, errors at overlap regions can propagate directly into counts and size distributions, making semantic segmentation less suitable as a stand-alone solution when per-bubble measurements are required [9, 13]. This motivates instance-level formulations in the presence of frequent overlap.

2.2.3 Deep learning: Mask R-CNN and instance segmentation

Instance segmentation addresses the overlap problem by predicting one mask per bubble, which aligns directly with downstream objectives such as counting and per-bubble size statistics [9]. Mask R-CNN is a common baseline in the multiphase-flow literature and has been adapted to dense sub-millimeter bubbly flows where overlap is severe [9]. Architecturally, Mask R-CNN extends Faster R-CNN by adding a parallel mask-prediction branch for each detected region of interest, enabling pixel-level instance masks rather than only bounding boxes [12, 24].

A practical challenge in experimental bubble images is the strong multi-scale nature of the scene: small bubbles and larger deformed bubbles may coexist in the same frame. Feature Pyramid Networks (FPNs) are therefore commonly paired with Mask R-CNN backbones to improve detection and segmentation across scales [17]. In addition, because segmentation outputs are often used for physical measurements rather than solely for visual correctness, several works incorporate post-processing or explicit reconstruction steps to improve boundary plausibility under occlusion and to reduce measurement bias in dense regimes [9]. More generally, transfer learning and fine-tuning from pretrained weights are frequently used to reduce the amount of domain-specific labeled data required for adaptation to new experimental conditions [11, 9].

2.2.4 Foundation models (SAM, SAM2)

Foundation segmentation models introduce a different paradigm: instead of training a task-specific model to directly output masks for a narrow domain, they provide a promptable segmentation interface that can be guided by sparse user inputs (e.g., points or boxes) and then refined interactively [16]. SAM2 extends this direction with an architecture designed to support segmentation across images and videos, enabling efficient interactive refinement in time-dependent data [23].

In the multiphase-flow context, recent work has explored combining SAM-style models with domain-specific refinements for bubble imagery, highlighting both the potential and the need for careful handling of challenging cases such as defocus and small bubbles [32]. Importantly for practical dataset creation, these models also enable model-assisted annotation workflows: annotation platforms such as Label Studio integrate SAM/SAM2-style interaction, allowing annotators to correct boundaries rapidly using prompt-guided masks rather than drawing every instance from scratch [4]. This capability is particularly relevant under limited annotation budgets and directly supports the labeling-efficiency strategies discussed later in Section 2.3 [31].

Quantitative comparison. Table 2.1 provides a quantitative snapshot of representative approaches discussed in this section, including semantic segmentation, instance segmentation with reconstruction, detection-plus-geometry pipelines, and promptable foundation-model workflows [26, 9, 11, 32, 30]. Because studies differ in datasets, operating regimes, and reporting practices, the metrics should be interpreted in context rather than as a direct ranking across works (see Section 2.4) [13].

Overall, the results reflect a consistent trade-off reported in the literature: semantic segmentation can perform well for phase separation but requires additional logic to separate touching bubbles, while instance-level methods better match bubble-by-bubble measurement goals but degrade as overlap and imaging artefacts increase [9, 13]. Detection-focused pipelines can achieve strong AP under overlap, yet do not provide pixel-accurate masks without additional segmentation stages [30]. Promptable foundation models can accelerate refinement and dataset creation, but reported failure cases in bubble imagery (e.g., defocus and small bubbles) motivate careful fine-tuning and human-in-the-loop workflows [32, 16, 23].

2.3 Training with limited annotations

Across multiphase-flow imaging studies, segmentation performance is often constrained less by model capacity than by the quantity and diversity of *reliable* labeled data available for training and validation [9, 13, 15]. This is particularly true for instance segmentation in dense multiphase flows, where accurate per-bubble masks are required to support downstream measurements (Section 2.1) and where the visual variability across experimental setups can be substantial [9, 28]. Consequently, a major line of work focuses on strategies that reduce dependence on large fully annotated datasets while improving robustness to domain shifts [15, 31].

Table 2.1: Quantitative comparison of representative segmentation approaches for bubbly-flow imagery (NR: not reported explicitly in the cited source).

Study	Architecture / type	Dataset context	IoU / mAP	Noted limitations
Seong et al. [26]	U-Net style (semantic)	High-speed experimental imagery	NR (reports overall correct classifications of 98.95%)	Overlap and touching bubbles still require additional handling; the paper notes remaining segmentation errors in challenging regions
Cui et al. [9]	Mask R-CNN + reconstruction	Dense, high-overlap sub-millimeter bubbles	mAP (and related metrics) > 0.95 for gas holdup $< 14.67\%$; metrics remain > 0.90 for gas holdup $< 20\%$	Performance degrades as holdup and overlap increase; difficult cases include uneven illumination, shadows, and blurred boundaries
Haas et al. [11]	Faster R-CNN + shape regression (BubCNN)	Experimental bubbly-flow imagery	mAP@0.5 = 0.84 (also reports F1 = 0.86)	Detection and shape quality depend on imaging conditions and generalization across setups can require adaptation
Xu et al. [32]	SAM-based bubble segmentation (BubSAM)	Challenging bubbly-flow imagery	Reports Precision/Recall/F1 (precision up to 97%; precision $> 99\%$ at low aeration) and a composite score σ ranging from 94.46% (low holdup) to 84.81% (higher holdup)	Reported failure modes include defocus and small bubbles; robustness depends on image quality and scenario
Wen et al. [30]	YOLOv5 detector + tracking (detection-focused)	Overlapping bubbles (tracking pipeline)	AP = 0.90; AP50 = 0.94 (Soft-NMS), 0.92 (no Soft-NMS)	Provides boxes not masks; pixel-accurate delineation would require an extra segmentation stage and tuning

2.3.1 The annotation bottleneck

Producing pixel-accurate instance masks for bubble imagery is time-consuming and inherently difficult in regimes with strong overlap, partial occlusion, and ambiguous boundaries [9, 13]. Unlike coarse labeling tasks, bubble segmentation commonly requires precise delineation because small boundary deviations can bias computed bubble areas and, in turn, skew bubble size distributions and other statistics [9]. Annotation cost is therefore not only a matter of time but also of consistency: dense scenes increase the likelihood of subjective decisions about split/merge cases, which can introduce label noise and limit the effective quality of the training signal [13].

From a learning perspective, the bottleneck is amplified by the need for diversity. Models trained on a narrowly sampled set of operating conditions or optical settings may not generalize well when illumination, camera parameters, or flow regimes change [9, 15]. As a result, research commonly targets methods that (i) expand the training distribution without proportional manual labeling effort and (ii) leverage existing pretrained representations to adapt effectively from small domain-specific datasets [15, 11].

2.3.2 Synthetic data and the domain gap

Synthetic image generation is a natural response to limited annotations because it can provide large volumes of perfectly labeled data at low marginal cost [15, 31]. In bubble segmentation, synthetic datasets can be constructed to span a range of bubble sizes, shapes, and densities, and can explicitly generate difficult configurations such as overlap and occlusion, which are costly to annotate in real images [9]. In practice, synthetic data is often used either to pretrain a model prior to adaptation on real imagery or to augment a small real dataset with additional labeled examples [15].

A persistent challenge is the *domain gap*: even visually plausible synthetic images may differ from experimental recordings in subtle but important ways, such as boundary texture, illumination non-uniformity, sensor noise, optical blur, or specular reflections [15]. These mismatches can lead to strong degradation when a model trained primarily on synthetic data is evaluated on real experimental images [15]. Consequently, synthetic pipelines are typically paired with techniques that encourage invariance to appearance changes (e.g., broader variability during generation or augmentation) and with *synthetic-to-real* adaptation via fine-tuning on a smaller set of real annotations [15, 31].

2.3.3 Transfer learning and fine-tuning

Transfer learning is widely adopted in segmentation because pretrained backbones provide robust low- and mid-level features that can be adapted with fewer labeled examples than would be required for training from scratch [11]. This is particularly relevant for instance segmentation frameworks such as Mask R-CNN, which are commonly initialized from large-scale datasets and then fine-tuned on domain-specific imagery [12, 24, 17]. In bubble segmentation studies, fine-tuning strategies are often chosen to balance stability and adaptation: for example, updating only higher layers initially (or using reduced learning rates for the backbone) can preserve general visual features while adapting the detection and mask heads to bubble-specific appearance and scale [11, 9].

For semantic segmentation settings, U-Net-style architectures are also frequently trained using transfer learning principles when available, and performance gains are reported even with limited labeled data [26]. More generally, transfer learning complements synthetic-data approaches: synthetic pretraining can teach task-specific priors (e.g., overlap handling), while subsequent fine-tuning on real data corrects for the domain gap [15, 31].

2.3.4 Data augmentation

Data augmentation is a practical mechanism to increase effective dataset diversity when annotation budgets are limited. Standard geometric transformations (e.g., flips, rotations, scaling) help reduce sensitivity to viewpoint and spatial configuration, while appearance-based perturbations (e.g., brightness/contrast shifts, blur, and noise injection) can mimic common variability in experimental imaging [15]. For bubble imagery, augmentation is especially useful to expose the model to plausible changes in illumination and boundary clarity that are known to affect segmentation reliability [9, 15].

Augmentation must nevertheless be designed with care. Excessive or unrealistic perturbations can introduce training examples that do not correspond to physically plausible observations, potentially harming generalization [15]. As a result, augmentation policies in multiphase-flow segmentation are typically motivated by known sources of variability in acquisition conditions and by the intended downstream measurements, prioritizing changes that preserve instance geometry while varying appearance in controlled ways [9, 15].

2.3.5 Model-assisted annotation workflows

An increasingly influential direction is to reduce manual labeling cost through interactive, model-assisted annotation. Foundation models such as SAM provide promptable segmentation in which a user supplies sparse guidance (e.g., points or boxes) and iteratively refines the resulting masks [16]. SAM2 extends this paradigm toward efficient segmentation across images and videos, which is particularly relevant when annotations must be propagated through sequences or when temporal consistency is desired [23]. In practice, annotation tools have begun integrating these capabilities; for example, Label Studio supports SAM-based workflows in which annotators correct and refine predicted masks rather than drawing them from scratch [4].

For bubble imagery, recent studies indicate that such interactive workflows can accelerate mask creation but may still require careful refinement in difficult cases such as small bubbles, defocus, and ambiguous boundaries [32]. A common pattern is iterative dataset growth: preliminary labels are produced with assistance from a model, corrected by a human, and then used to retrain or fine-tune the segmentation model, progressively improving both model performance and annotation efficiency [31]. This human-in-the-loop perspective is especially aligned with experimental research settings, where new operating conditions may continually introduce distribution shifts and require practical mechanisms for rapid dataset expansion [15].

2.4 Datasets and evaluation

2.4.1 Available datasets

The performance and generalizability of segmentation models for multiphase-flow imaging are strongly shaped by the availability of labeled datasets. In dense bubbly flows, pixel-accurate *instance* masks

are particularly costly to obtain, which often limits dataset scale and diversity, and explains why many studies rely on comparatively small annotated sets, synthetic data generation, or a combination of both [9, 13, 15]. Table 2.2 summarizes representative datasets and dataset-building strategies, highlighting the typical supervision level, reported scale, metrics used for evaluation, and whether the data are publicly accessible [9, 13, 28, 15, 32].

Table 2.2: Representative datasets and dataset-building strategies used in bubble segmentation studies (NR: not reported explicitly in the cited source).

Work	Data	Output	# Img	# Inst.	Metrics ported	re-	Public?	Notes (context + relevance)
Cui et al. [9]	Exp.	Instance recon.	+ 100	>20k bubbles	Prec., mAP, F1, IoU	Rec.,	Not stated	Dense sub-millimeter bubbly flow with strong overlap; combines Mask R-CNN-style instance segmentation with shape reconstruction to improve measurement reliability in high-holdup regimes.
Hessenkemper et al. [13]	Exp. synth.	+ Instance ID / segm.	800 (GT) + 1000 (synth.)	24.4k (GT bubbles)	AP (IoU-based), detection/segm. errors		Yes	Comparative benchmark including classical and learning-based methods; reports an annotated dataset and synthetic compositions; provides open repositories (dataset/code/models) (BubMask).
Soibam et al. [28]	Exp. (high-speed)	Instance masks	600 (80/20 split)	2237 (val); total NR	Prec., AP@0.5, mAP@[0.5:0.95]	Rec.,	On request	Transfer learning for boiling/high-speed imagery; reports detection-style metrics and explicit splits; availability stated as “upon request”.
Homan & Deen [15]	Synth. public real	+ Instance masks / pseudo-labels	100 (synth.) + 10k (pseudo)	10k (synth. bubbles)	mAP@0.5, mAP@[0.5:0.95]		Partial	“Shoestring” pipeline: synthetic scenes (100 images, 100 bubbles/image) and large-scale pseudo-labeling on a publicly available real dataset; code released, while derived labels depend on the underlying dataset terms.
Xu et al. (BubSAM) [32]	Exp.	Masks + recon.	NR	~8000 bubbles/-condition	Prec., composite score σ ; recon.-related errors	Rec., F1;	On request	SAM-based bubble segmentation with reconstruction; reports results across multiple holdup conditions; full dataset not publicly released (data availability: on request).

Overall, Table 2.2 illustrates two recurring patterns: (i) fully annotated experimental datasets remain relatively limited compared with general-purpose vision benchmarks, and (ii) public availability is inconsistent, which complicates reproducible comparison across studies [13, 9, 15]. These constraints motivate training strategies that maximize data efficiency and robustness, discussed in Section 2.3 [15, 31].

2.4.2 Evaluation metrics and practices

Evaluation protocols in bubbly-flow segmentation typically combine pixel-level overlap measures with instance-level detection-style measures, reflecting that the downstream objective is often bubble-by-bubble measurement rather than only visual plausibility [9, 13]. For semantic segmentation, the most common overlap metric is Intersection over Union (IoU), often complemented by Dice/F1-type overlap

scores and pixel-level precision/recall, which are sensitive to boundary errors and merged/split regions [9, 32]. When instance masks are required, studies frequently adopt Average Precision (AP) and mean Average Precision (mAP) computed by matching predicted instances to ground-truth instances under IoU criteria (e.g., AP@0.5 and mAP over multiple IoU thresholds), because these measures jointly reflect missed detections, false positives, and mask quality [13, 28, 9].

Because segmentation is commonly used to compute physical statistics, several works also report measurement-oriented indicators derived from masks, such as bubble counting accuracy and errors in size-related quantities (e.g., area or equivalent diameter) and their distributions [9, 32]. Moreover, given the strong dependence on operating regime and imaging setup, evaluation is often stratified by conditions (e.g., increasing gas holdup) or performed across distinct acquisition settings to assess generalization, and this regime-aware reporting is essential for interpreting whether a model is reliable for experimental use [9, 15, 32].

2.5 Research gaps

Although modern segmentation architectures and promptable foundation models have improved performance and usability, reliable bubble segmentation in experimental multiphase imaging remains limited by a small set of practical gaps.

(1) Robustness across regimes and setups is still not guaranteed. Reported results are often tied to specific operating conditions and imaging configurations, and performance can degrade under increased overlap, blur/defocus, and illumination variability [9, 15, 32]. This makes it difficult to assume that a model trained or validated in one setting will remain reliable when experimental conditions change.

(2) Data efficiency remains a bottleneck for instance-level supervision. Bubble-by-bubble measurements favor instance segmentation (Section 2.1), yet dense scenes make pixel-accurate instance labeling costly and inconsistent, and public datasets with comparable annotation standards remain limited [9, 13, 28]. As a consequence, methods that reduce dependence on extensive manual labeling—synthetic data, transfer learning, and augmentation—are widely used but require careful design to be effective [15, 31].

(3) Evaluation is not always aligned with measurement impact. IoU and AP/mAP are standard, but they do not necessarily reveal whether segmentation errors introduce unacceptable bias in downstream quantities (e.g., counts and size distributions), especially in dense regimes where split/merge errors dominate [9, 13]. More consistent reporting of regime-stratified performance and measurement-oriented error indicators would improve comparability and experimental trustworthiness [9, 32].

(4) The practical value of interactive model-assisted annotation needs clearer quantification. SAM/SAM2-style workflows can accelerate annotation, but bubble imagery includes difficult cases (small bubbles, defocus, ambiguous edges) where interactive refinement is still required, and the resulting time savings and performance gains are not yet consistently quantified across conditions [16, 23, 32, 4].

Implications for this dissertation. These gaps motivate an experimental study centered on instance segmentation models and data-efficient training, including synthetic-to-real fine-tuning and augmentation, together with a quantitative assessment of model-assisted annotation workflows for generating reliable instance masks in experimental images [15, 31, 16, 23, 4].

Chapter 3

Methodology

3.1 Overview of the research workflow

This dissertation follows an end-to-end workflow that connects dataset preparation, model training, evaluation, and model-assisted annotation into a reproducible pipeline. The goal is to study bubble *instance segmentation* in experimental multiphase-flow images under realistic constraints, namely limited labeled data and strong variability across experimental conditions.

To avoid ambiguity about how the different datasets, training stages, and evaluation activities connect, Figure 3.1 summarizes the complete methodology as a single pipeline. In particular, it highlights (i) the parallel use of a synthetic COCO-formatted dataset and an experimentally annotated COCO dataset, (ii) the four training conditions repeated for each architecture (synthetic training, real baseline training, synthetic-to-real transfer learning, and transfer learning with augmentation), and (iii) the two outcome streams of the work: quantitative model evaluation on the experimental test split and a separate annotation-efficiency study using model-assisted labeling.

The workflow is organized as follows:

1. **Data preparation and label standardization.** Two types of data are considered: (i) synthetic bubble images used to obtain large-scale training data with complete instance masks, and (ii) experimental images used for model evaluation and for fine-tuning. All supervised experiments use a COCO-style instance segmentation format [18]. The synthetic dataset is provided in this format [22], while experimental annotations are created in Label Studio and exported/standardized into the same structure [2].
2. **Baseline model selection and synthetic training.** A set of deep-learning instance segmentation baselines is trained on synthetic data, with Mask R-CNN variants playing a central role [12]. The approach builds on the two-stage detection paradigm introduced by Faster R-CNN [24] and employs feature pyramid backbones when applicable to support multi-scale representation [17]. Model configuration is controlled via YAML experiment files to enable systematic comparisons

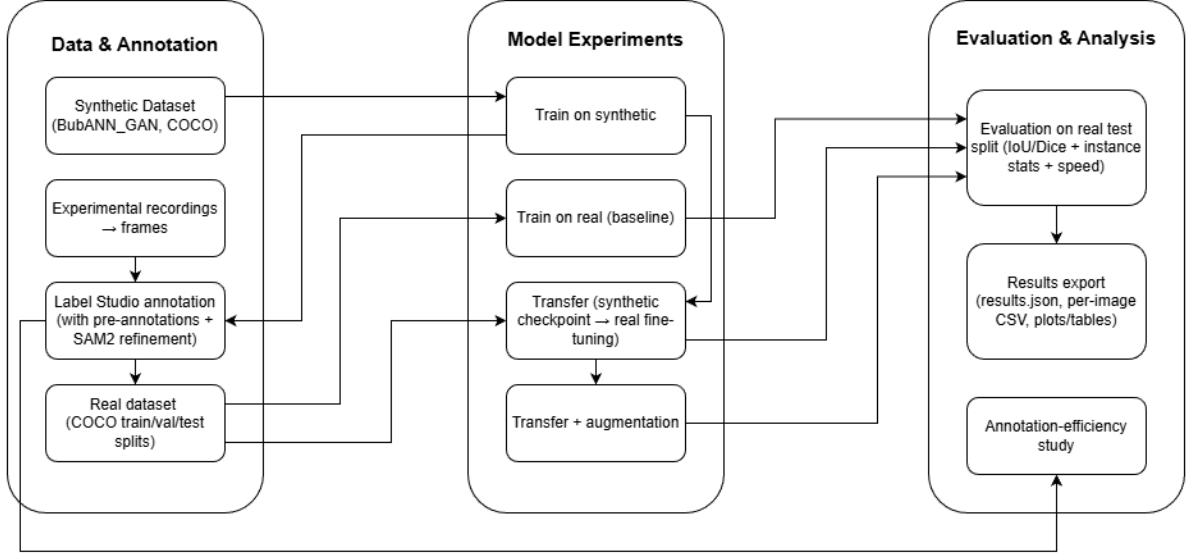


Figure 3.1: End-to-end research workflow adopted in this dissertation: dataset preparation and annotation, repeated model experiments (synthetic training, real baseline training, synthetic-to-real transfer learning, and transfer learning with augmentation), evaluation on a held-out experimental test split, and an annotation-efficiency study using model-assisted labeling.

across architectures and training settings, and reference implementations are based on PyTorch/-TorchVision [21, 5].

3. **Experimental annotation and model-assisted labeling.** To reduce the cost of producing pixel-accurate instance masks in experimental images, the workflow includes model-assisted labeling in Label Studio [1, 2]. In this loop, pre-annotations (generated automatically) are refined by the annotator, consistent with semi-automatic annotation strategies described in the computer-vision literature [31]. Interactive refinement is incorporated using SAM2 to accelerate boundary correction through prompt-guided mask editing [23, 4]. An annotation-efficiency study compares manual-only annotation against workflows with pre-annotations and SAM2-assisted refinement.
4. **Real-domain training and synthetic-to-real transfer learning.** For each architecture, four training conditions are considered: (i) training on synthetic data; (ii) training on the annotated real dataset (real baseline); (iii) synthetic-to-real transfer learning initialized from the *corresponding synthetic-trained checkpoint for that architecture* (not a single globally “best” model); and (iv) transfer learning with training-time data augmentation enabled. The transfer-learning stage follows a practical protocol: load the synthetic checkpoint, optionally freeze the backbone for a small number of epochs to stabilize adaptation, then unfreeze and continue fine-tuning with a reduced backbone learning rate. This is consistent with established transfer learning practice and is particularly relevant under limited annotation budgets [20, 15]. Data augmentation is treated

as a separate factor in the experimental design and follows standard deep-learning augmentation principles [27].

5. **Model evaluation on experimental test data and results export.** After training, performance is evaluated on held-out experimental test images using a unified protocol that reports (i) pixel-level overlap metrics (e.g., IoU and Dice, with precision and recall), (ii) instance-level bubble statistics (e.g., bubble counts and area summaries, including counting error), and (iii) indicative inference throughput (average inference time and frames per second). Results are exported in a structured format to support cross-experiment analysis.
6. **Experiment tracking and reproducibility.** Experiments are managed through consistent configuration files and structured outputs. Implementation is based on PyTorch and TorchVision, providing stable reference implementations for detection and instance segmentation models [21, 5].

3.2 Data sources and label formats

This dissertation uses two complementary data sources: a synthetic dataset with fully available instance masks and an experimental dataset obtained from multiphase-flow recordings. The synthetic data supports large-scale supervised training under controlled conditions, while the experimental data is required to evaluate generalization and to study synthetic-to-real fine-tuning under realistic artefacts such as overlap, reflections, and illumination variability.

3.2.1 Synthetic dataset

The synthetic training data is based on the **BubANN_GAN** dataset [22], which provides bubble instances with pixel-level annotations in a COCO-style segmentation format. The version used in this dissertation contains approximately **1.5k images** and is distributed with predefined **training, validation, and test** splits.

The dataset follows the common COCO-segmentation directory structure, where each split contains the images and a corresponding `_annotations.coco.json` file:

```
BubANN_GAN.v1i.coco-segmentation/
train/
  _annotations.coco.json
  *.jpg
val/
  _annotations.coco.json
  *.jpg
test/
  _annotations.coco.json
```

*.jpg

No additional transformations are permanently applied to the dataset on disk. Instead, resizing and augmentations (when enabled) are applied *on-the-fly* during training to ensure consistent preprocessing across synthetic and experimental data (Section 3.4).

3.2.2 Experimental dataset and frame selection

Experimental bubble images were provided in collaboration with CEFT/FEUP as video recordings from multiphase-flow experiments. From these videos, a total of **1,397** individual frames were extracted to form an initial pool of candidate images.

Only a subset of these frames was selected for annotation. This selection was performed to maximize visual diversity (for example, different bubble sizes, overlap regimes, and lighting conditions) and to reduce redundancy caused by near-duplicate frames originating from the same short time intervals. As a result, **243 frames** were chosen as a first representative set for annotation. This curated set was treated as an initial supervised dataset that could be expanded in future work; however, due to time constraints, the final experimental dataset used in this dissertation is based on these **243 annotated images**.

3.2.3 Annotation tool and export formats

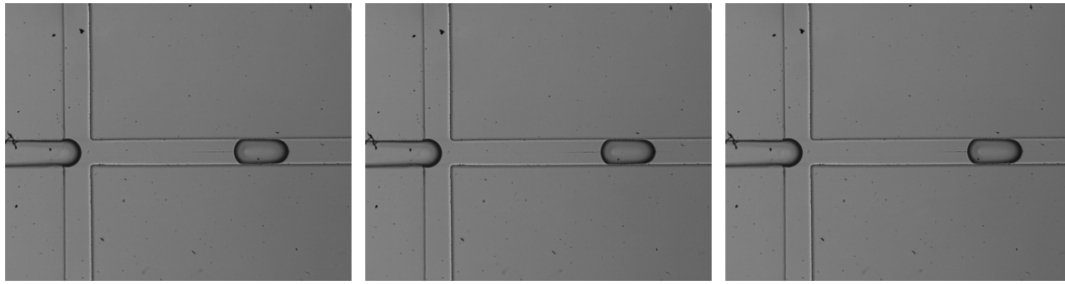
All manual ground truth annotation was performed in **Label Studio** [2]. Label Studio was used both for fully manual annotation and for model-assisted refinement workflows (described later in Section 3.5). For training and evaluation, annotations were exported to a COCO-compatible instance segmentation format [18], which represents each bubble instance through: (i) a per-instance segmentation (polygon-based representation), (ii) a bounding box, and (iii) an image-level identifier linking instances to the source frame.

In addition to COCO export, Label Studio task JSON was also used when operating with model-assisted annotation and interactive refinement, because it supports importing pre-annotations as prediction layers and refining them within the annotation interface [4].

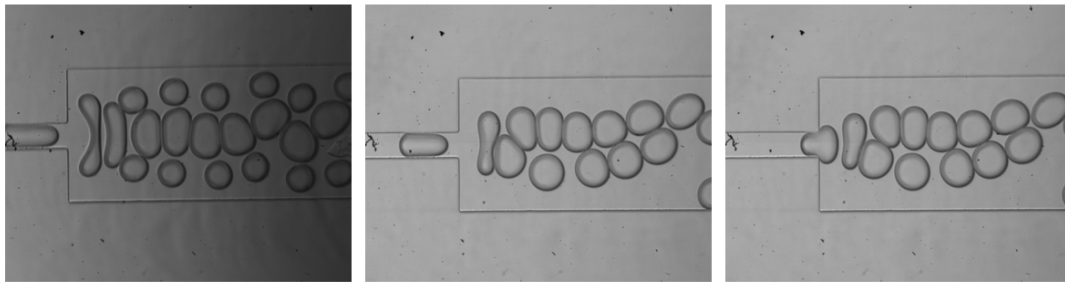
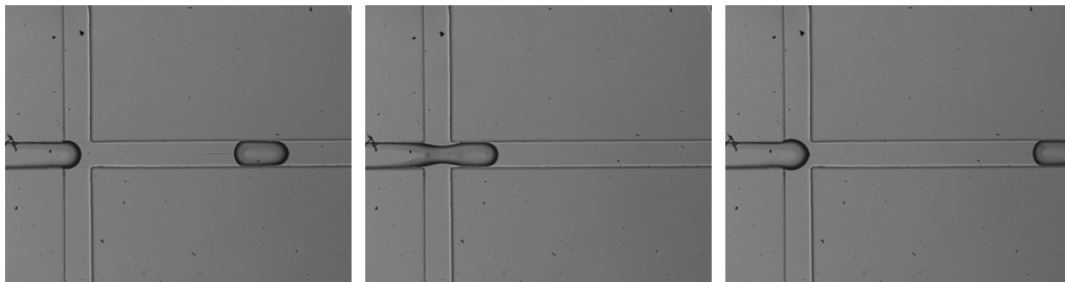
3.2.4 Dataset standardization and split protocol

To ensure a consistent training interface across experiments, both datasets were organized into the same COCO-segmentation directory convention (`train/val/test`), each containing images and an `_annotations.coco.json` file.

The synthetic dataset is already provided in this format and is already split. For the experimental dataset, the annotated export is post-processed to (i) normalize image file names, (ii) reindex image and annotation identifiers, and (iii) generate reproducible train/validation/test splits with a fixed seed. In addition, images are grouped by filename prefix before splitting to reduce leakage between visually similar frames (for example, consecutive frames extracted from the same original sequence). The split ratios used in this dissertation are 70% training, 20% validation, and 10% test.



(a) Examples from the extracted pool (redundancy across adjacent frames).



(b) Examples from the selected set (higher visual diversity).

Figure 3.2: Illustration of the frame-selection step used to build the annotated experimental dataset.

3.2.5 Preprocessing and training-time resizing

Images are stored in their original resolution in the dataset folders. During training and evaluation, a fixed-size resize is applied as part of the input pipeline so that all models receive consistent tensor dimensions and comparable augmentations. Appearance augmentations (e.g., brightness/contrast changes, blur, and noise-like effects) are applied only during training and only when enabled in the experiment configuration, ensuring that validation and test results reflect performance on unaugmented experimental imagery.

Source	Quantity	Supervision	Role in this dissertation
BubANN_GAN [22]	~1.5k images (pre-split)	Instance masks (COCO)	Large-scale supervised training and baseline comparison under controlled synthetic conditions
Experimental frames (CEFT/FEUP)	1,397 extracted; 243 annotated	Instance masks (Label Studio → COCO)	Real-domain evaluation, fine-tuning (synthetic-to-real transfer learning), and annotation workflow analysis

Table 3.1: Datasets used in this dissertation and their role in the experimental workflow.

3.3 Segmentation models evaluated

This dissertation evaluates deep-learning *instance segmentation* models for bubble delineation in experimental multiphase-flow images. The primary objective is to obtain *pixel-accurate* masks for each individual bubble under overlap and variable imaging conditions, enabling downstream measurements such as bubble count and size statistics. Because these measurement goals require bubble-by-bubble separation, the methodology prioritizes instance segmentation models that directly output one mask per bubble, rather than detection-only pipelines that only provide bounding boxes.

3.3.1 Model family and evaluated variants

The quantitative experiments in this dissertation focus on the Mask R-CNN family, which is widely used as an instance segmentation baseline and has been successfully applied to dense bubbly-flow imagery with strong overlap [9]. Mask R-CNN combines region-based detection with per-instance mask prediction [12, 24] and is commonly paired with Feature Pyramid Networks (FPNs) to improve multi-scale performance [17], which is particularly relevant for scenes containing both small and large bubbles in the same frame.

To study the trade-off between backbone capacity and practical performance in this domain, three TorchVision Mask R-CNN variants are evaluated throughout the experimental protocol:

- `maskrcnn_resnet50_fpn` (ResNet-50 backbone with FPN),
- `maskrcnn_resnet50_fpn_v2` (updated ResNet-50-FPN variant in TorchVision),
- `maskrcnn_resnet101_fpn` (higher-capacity ResNet-101 backbone with FPN).

Although the codebase includes support for semantic segmentation baselines (e.g., U-Net-style models), the controlled comparisons and results reported in this dissertation are centered on instance-level mask prediction, because semantic masks often merge overlapping bubbles unless additional separation stages are introduced.

3.3.2 Mask R-CNN as the primary baseline

Mask R-CNN extends Faster R-CNN by adding a parallel mask head for each detected region of interest, thereby producing instance-level masks rather than only class labels and bounding boxes [12, 24]. Concretely, the model (i) proposes candidate regions, (ii) predicts a class label and bounding-box refinement for each region, and (iii) predicts a binary mask for each selected region. This design is well suited to dense bubbly-flow imagery, where bubbles frequently touch or overlap and must be separated as distinct instances [9].

A key practical aspect for bubble imagery is **multi-scale representation**. Because the bubble population in a given frame can include both small and large instances, this work uses Mask R-CNN variants equipped with FPN backbones to improve robustness across scales [17]. In the context of multiphase experiments, this reduces the likelihood that small bubbles are missed while larger deformed interfaces remain well delineated.

3.3.3 Configuration and implementation choices

All models are trained and evaluated within a unified PyTorch-based pipeline using TorchVision model implementations and pretrained weights when applicable [21, 5]. Experiments are controlled through YAML configuration files that define the model variant, training hyperparameters, augmentation switches, output structure, and evaluation thresholds. This setup promotes reproducibility and supports fair comparison by keeping dataset splits and evaluation logic fixed across runs.

Output definition and class convention. The task is formulated as *binary* instance segmentation with a single foreground class (*bubble*). Following the standard COCO convention [18], the background is treated implicitly, so the model is configured with `num_classes = 2` (background + one foreground class). Each predicted bubble instance is represented by a mask, a bounding box, and a confidence score.

Fixed post-processing for comparable measurements. To ensure consistent comparison across models and training regimes, inference outputs are filtered using fixed thresholds defined in the experiment configuration: a confidence-score threshold, a mask-binarization threshold, and a minimum-area filter to remove small spurious detections. These thresholds are kept identical across runs so that differences in bubble count and size statistics reflect model behavior rather than tuning changes.

3.4 Training pipeline and experiment configuration

To ensure that experiments are reproducible and directly comparable, all training and evaluation runs are parameterized through YAML configuration files. Each configuration specifies dataset paths, the model variant, optimization hyperparameters, optional data augmentation switches, output directories, and fixed evaluation thresholds. This configuration-driven design ensures that changes between runs are

explicit and traceable, while keeping dataset splits and evaluation logic identical across model comparisons.

3.4.1 Configuration-driven experiments (YAML)

Each YAML file is organized into logically separated blocks:

- `experiment`: experiment name and random seed.
- `model`: model identifier and task definition (`num_classes`).
- `dataset`: paths to `train`, `val`, and `test` splits (COCO-style).
- `train`: hyperparameters (batch size, learning rate, scheduler, workers) and early stopping (`patience`).
- `augmentation`: preprocessing (e.g., `resize`) and optional random augmentations.
- `logging`: TensorBoard log directory (and a `use_tensorboard` switch when supported).
- `output`: base directory where checkpoints and logs are stored.
- `eval`: post-processing thresholds used during evaluation (score threshold, minimum area, mask threshold, and optional visualization seed).
- `transfer` (transfer runs only): checkpoint initialization and adaptation controls (`init_checkpoint`, `strict_load`, `freeze_backbone_epochs`).

Listing 3.1 shows an example configuration used for standard training on the synthetic dataset.

```
1 experiment:
2   name: maskrcnn_resnet50_fpn_synth
3   seed: 42
4
5 model:
6   type: maskrcnn_resnet50_fpn
7   num_classes: 2
8   pretrained_backbone: true
9
10 dataset:
11   train: ../../BubANN_GAN.v1i.coco-segmentation/train
12   val:   ../../BubANN_GAN.v1i.coco-segmentation/valid
13   test:  ../../BubANN_GAN.v1i.coco-segmentation/test
14
15 train:
16   batch_size: 2
17   epochs: 50
```

```
18   lr: 0.001
19   backbone_lr_factor: 0.1
20   momentum: 0.9
21   weight_decay: 0.0005
22   lr_step_size: 7
23   gamma: 0.1
24   num_workers: 4
25   print_freq: 20
26   patience: 7
27
28 augmentation:
29   enabled: false
30   resize: [512, 512]
31   hflip_p: 0.0
32   strong_color_jitter: false
33   blur: false
34   intensity_shift: false
35   rotation: false
36
37 logging:
38   use_tensorboard: true
39   log_dir: runs/
40
41 output:
42   dir: outputs/
43
44 eval:
45   score_thresh: 0.5
46   min_area: 50
47   mask_thresh: 0.5
```

Listing 3.1: Example YAML configuration used to drive a standard training run on the synthetic dataset.

3.4.2 Default configuration values and rationale

To ensure fair comparisons, the Mask R-CNN experiments use a common set of training and evaluation parameters. All runs use an input resolution of 512×512 pixels and SGD optimization with momentum and weight decay, following standard torchvision reference practice for detection and instance segmentation training [6]. Learning-rate scheduling uses StepLR with a multiplicative decay factor $\gamma = 0.1$, which is widely used in PyTorch reference training recipes [7]. The base learning rate is set conservatively (10^{-3}) because training is performed with batch size 2 under limited GPU memory, and because early transfer-learning adaptation can be unstable when pretrained features are updated too aggressively, the fine-tuning stage uses conservative updates (backbone LR scaling and/or temporary freezing), as commonly recommended in transfer-learning practice [29].

The batch size was fixed to 2 across experiments to remain within GPU memory constraints in the available execution environment, while preserving stable training dynamics. Training runs are capped at 50 epochs with early stopping (`patience=7`) to prevent unnecessary computation once validation performance plateaus. Learning-rate scheduling uses StepLR with `lr_step_size=7` and `gamma=0.1`, which progressively reduces the learning rate to support convergence in later epochs.

For transfer learning, the fine-tuning stage loads a synthetic-trained checkpoint and optionally freezes the backbone for the first `freeze_backbone_epochs=3` epochs. This warm-start strategy reduces instability in the early adaptation steps and encourages the heads to align to the target domain before full backbone updates are enabled.

Evaluation uses fixed post-processing thresholds (`score_thresh=0.5`, `mask_thresh=0.5`, `min_area=50`) so that differences in bubble count and mask overlap metrics reflect model behavior rather than per-run threshold tuning. These thresholds affect both the filtered instance set used for bubble statistics and the union-mask used for semantic overlap metrics, and are therefore held constant across comparisons.

Table 3.2: Default training and evaluation parameters used across Mask R-CNN experiments.

Parameter	Default value
Input resize	512×512
Batch size	2
Epochs (max)	50
Optimizer	SGD (momentum 0.9, weight decay 5×10^{-4})
Learning rate	10^{-3}
Backbone LR factor	0.1
Scheduler	StepLR (step=7, $\gamma=0.1$)
Early stopping patience	7
Transfer backbone freeze	3 epochs (transfer runs)
Eval score threshold	0.5
Eval mask threshold	0.5
Eval min area	50 pixels

3.4.3 Training regimes and execution

The experimental protocol repeats the same training logic across four regimes for each evaluated model: (i) **synthetic training**, (ii) **real baseline training**, (iii) **synthetic-to-real transfer learning**, and (iv) **transfer learning with augmentation**. To ensure fair comparisons, preprocessing settings, optimizer and scheduler configuration, early-stopping criteria, and evaluation thresholds were kept fixed across experiments; only the dataset domain (synthetic vs. real), the use of checkpoint initialization for transfer learning, and the activation of augmentation were varied.

Regimes (i) and (ii) are standard training runs configured for different dataset domains (synthetic vs. real) through the YAML dataset paths, while keeping preprocessing, optimization, and evaluation thresholds unchanged. Regimes (iii) and (iv) additionally include a `transfer` block specifying a

synthetic-trained checkpoint used to initialize fine-tuning on the real dataset; regime (iv) further enables training-time augmentation.

Transfer learning is implemented by loading `transfer.init_checkpoint` (optionally with strict state-dict matching), and optionally freezing the backbone for a small number of epochs using `transfer.freeze_backbone_epochs`. After the freeze period, the backbone is unfrozen and training continues with a reduced backbone learning rate via the `backbone_lr_factor`. This follows common transfer-learning practice for adapting pretrained features to a new domain under limited labeled data [20].

3.4.4 Dataset interface, preprocessing, and augmentation

All experiments use a COCO-style instance segmentation structure [18], where each sample contains an image and a set of instance annotations (segmentation + bounding box + metadata). During loading, polygon and/or RLE segmentations are decoded into binary instance masks and paired with bounding boxes in `XYXY` format. To preserve TorchVision conventions for detection models, category indices are mapped so that the foreground class `bubble` is always label 1, with background treated implicitly as label 0.

A fixed preprocessing stage is always applied to ensure consistent tensor shapes: images are resized to a configured resolution (512×512 in the default configuration), converted to `float32` (range-scaled), and converted into TorchVision-compatible tensor containers so that image and target transforms remain synchronized [21, 5]. Invalid boxes created by geometric transforms are filtered after transformation to avoid corrupted samples during training.

Random augmentation is controlled by `augmentation.enabled`. When enabled for training, the pipeline can apply horizontal flipping, optional color jitter, optional blur, and a sharpness adjustment (controlled by `intensity_shift`). Rotation is intentionally not applied for Mask R-CNN runs in this pipeline to avoid inconsistencies with region-based instance targets; it is reserved for semantic models only. This controlled augmentation design follows standard recommendations for improving generalization while keeping target consistency under geometric transforms [27].

3.4.5 Optimization, checkpointing, and reproducibility

Optimization uses SGD with momentum and weight decay. To stabilize fine-tuning of pretrained features, the optimizer is built with two parameter groups: detection/mask heads use the base learning rate, while backbone parameters use `lr × backbone_lr_factor`. Learning-rate scheduling follows a StepLR schedule configured by `lr_step_size` and `gamma`. When training on GPU, automatic mixed precision is enabled through gradient scaling to reduce memory use and improve throughput.

Validation is performed at every epoch. For Mask R-CNN runs, model selection follows COCO-style instance segmentation evaluation, and the best checkpoint is selected by the segmentation AP

averaged over IoU thresholds (AP@[0.50:0.95]) [18, 5]. Early stopping is applied when the validation criterion does not improve for `patience` epochs, and the best model parameters are stored as `model_best.pth`.

Reproducibility is promoted by explicit seeding of Python, NumPy, and PyTorch RNGs (including CUDA when available) at the start of each run. Each experiment also records the full YAML configuration in the run logs (TensorBoard text summary), ensuring traceability of settings across repeated training sessions [21].

Final evaluation metrics and exported artifacts are described in Section 3.6.

3.5 Annotation-efficiency study: synthetic pre-annotations and SAM2-assisted refinement

After completing the synthetic training workflow described in Section ??, a Mask R-CNN model with a ResNet-50-FPN backbone trained on the BubANN_GAN dataset was used to generate pre-annotations for a subset of real experimental images. This section defines a controlled protocol to quantify how (i) synthetic pre-annotations and (ii) SAM2-assisted interactive refinement affect annotation effort in Label Studio, using the tool’s built-in time tracking (`lead_time`).

3.5.1 Motivation and background

Producing pixel-accurate instance masks for bubbly multiphase flows is a well-known bottleneck in dataset creation due to high bubble densities, overlap/occlusion, variable illumination, and ambiguous thin-film interfaces. Substantial manual effort has been reported in prior studies, often requiring extensive interactive refinement to obtain reliable instance masks [9, 11, 10].

Label Studio supports semi-automated labeling through model-in-the-loop workflows, where predictions are imported and corrected by an annotator [2, 1]. More recently, foundation segmentation models such as SAM [16] and SAM2 [23, 3] have enabled prompt-based interactive mask refinement, and SAM-style tools are now integrated into Label Studio workflows [4]. Building on these ideas, the present study quantifies the time savings achieved by combining synthetic Mask R-CNN pre-annotations with SAM2-assisted refinement.

3.5.2 Study design

Image selection. From the full pool of 1,397 experimental frames (Subsection 3.2.2), a subset of 14 images was selected to cover a range of bubble densities, magnifications, contrast levels, and interface complexity. The same 14 images are used in all annotation modes to enable paired, within-image comparisons of annotation time. Figure 3.3 illustrates the targeted variability.

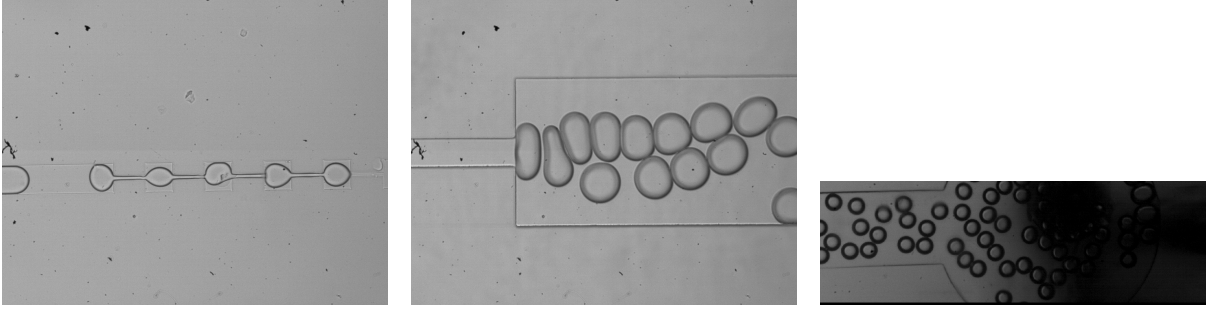


Figure 3.3: Representative examples from the 14 selected frames, illustrating variability in bubble density, magnification, and contrast targeted in the annotation-efficiency study.

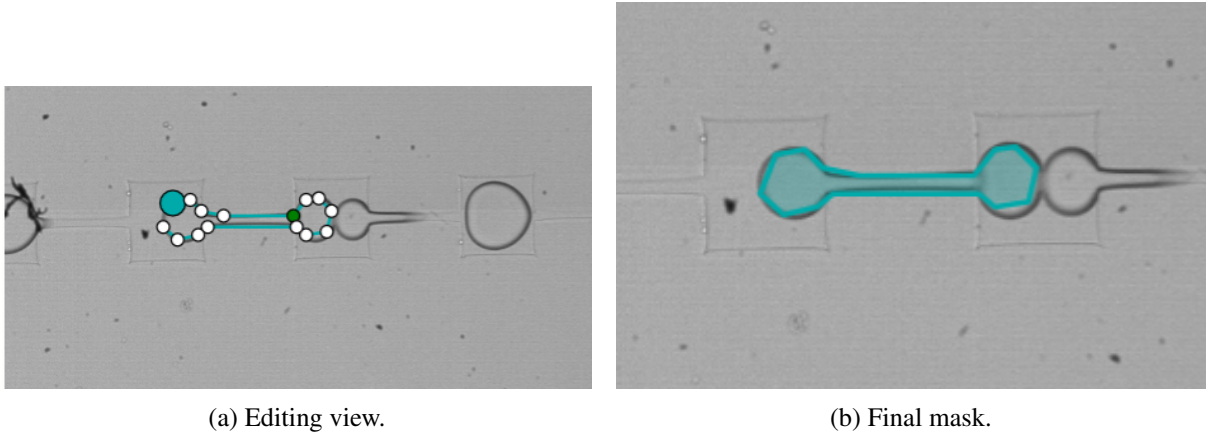


Figure 3.4: Manual polygon-based annotation in Label Studio (Mode A), used when bubble contours are irregular or require careful contour.

Annotation modes. Three annotation modes were defined to isolate the effect of automation:

Mode A (manual baseline). All bubble masks were created from scratch in Label Studio using manual tools (brush and polygon). This reflects conventional manual annotation practice reported in bubbly-flow segmentation work [33, 26]. The brush tool is efficient for near-elliptical bubbles, while the polygon tool provides vertex-level control for irregular boundaries and strong overlap. Figures 3.4 and 3.5 illustrate typical baseline cases, while Figure ?? shows a difficult low-contrast example where manual effort increases substantially.

Mode B (synthetic pre-annotations). A Mask R-CNN ResNet-50-FPN model trained on BubANN_GAN was used to generate instance mask predictions for the same 14 experimental images. Predictions were filtered using fixed score and minimum-area thresholds, converted into polygon annotations (via contour extraction / polygonization of binary masks), and imported into Label Studio as a prediction layer. The annotator then corrected missed instances, removed false positives, and refined inaccurate boundaries instead of drawing all masks from scratch. This is consistent with semi-automatic annotation strategies commonly used for detection/segmentation datasets [31, 2].

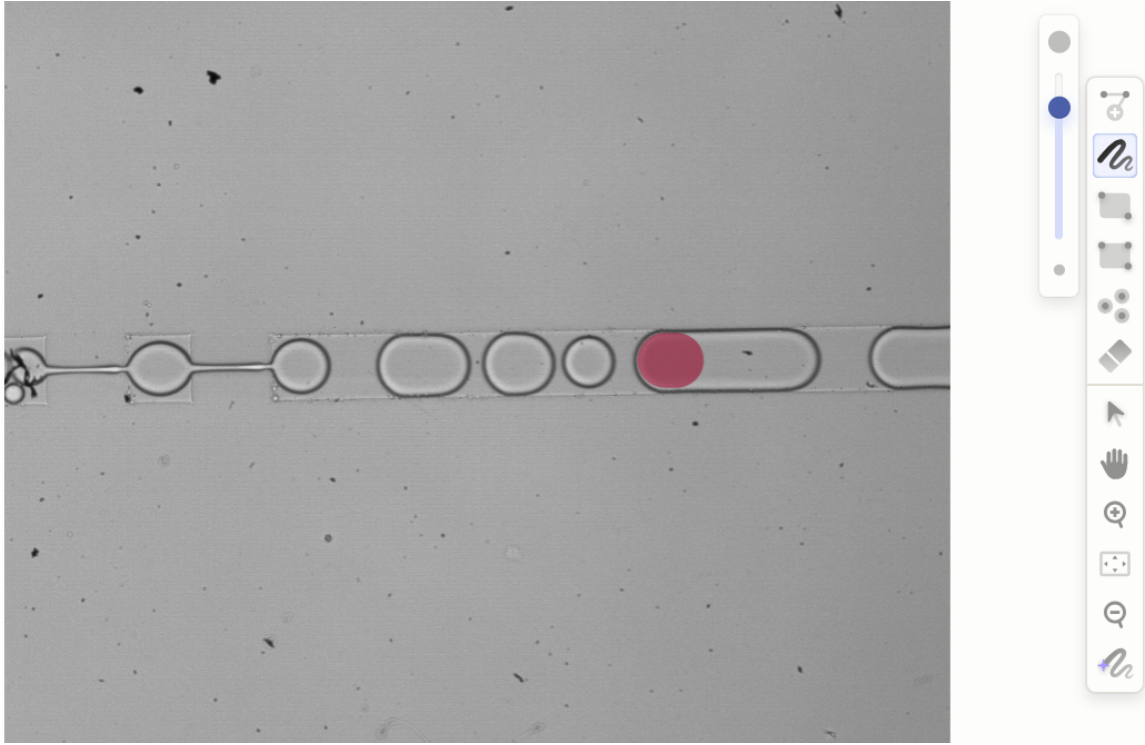


Figure 3.5: Manual brush-based annotation in Label Studio (Mode A), used for regular and approximately circular/elliptical bubbles.

Figure 3.6 shows a case requiring minimal intervention, while Figures 3.7–3.8 illustrate failure patterns (missed bubbles, over-segmentation, or boundary drift) that require correction.

Mode C (pre-annotations + SAM2 refinement). Mode C starts from the same Mode B prediction layer but enables SAM2 within Label Studio to accelerate interactive refinement. SAM2 refinement is triggered using point prompts (click inside/outside a bubble) and/or box prompts around regions of interest, and its predicted masks are used to replace coarse or incorrect regions before final manual adjustments. This leverages prompt-based segmentation to speed up boundary corrections in difficult regions [16, 23, 4]. Figures ??–?? illustrate typical interactions and a challenging example requiring refinement.

Annotation completion criterion. An annotation task was considered complete when **all visible bubble instances** in the frame were delineated with a closed instance mask. Bubbles were annotated whenever a visually discernible interface was present (including partially visible bubbles at the image border). Ambiguous or low-contrast regions were annotated according to the most consistent visible boundary, keeping the same interpretation across modes (A/B/C) to ensure that timing comparisons reflect tool support rather than changes in labeling strictness.

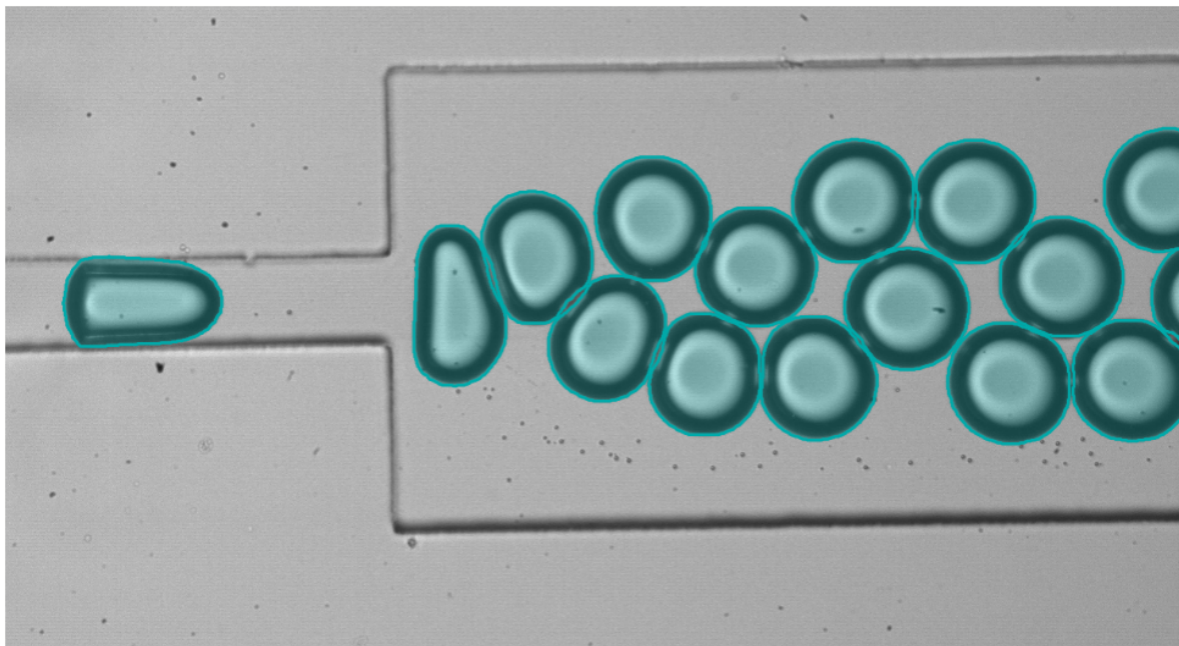
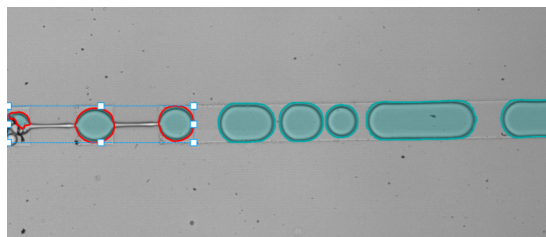
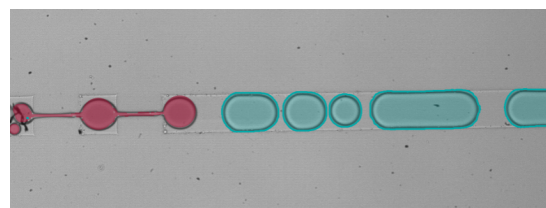


Figure 3.6: Mode B example where synthetic pre-annotations closely match the target masks and require only minor edits.



(a) Removed.



(b) Corrected.

Figure 3.7: Mode B correction examples: removal of incorrect instances, and final corrected annotation.

3.5.3 Timing signal and derived efficiency metrics

Label Studio records annotation effort per task using the `lead_time` field (seconds between opening and submitting a task) [2]. For each mode (A/B/C), `lead_time` was exported for the 14 images. Because `lead_time` can include short idle periods (e.g., pauses during annotation), results are interpreted as practical end-to-end effort under the same tool and interface.

From each exported task, the final number of annotated bubble instances was computed, and the following per-image indicators were derived for each mode: (i) total seconds per image (`lead_time`), (ii) number of bubbles, (iii) seconds per bubble, and (iv) bubbles per minute. Outputs were stored as long-format (image $\times$ mode) and wide-format (one row per image, columns per mode) CSV tables, and summary statistics (mean/median/std) were computed per mode. Finally, per-image percentage

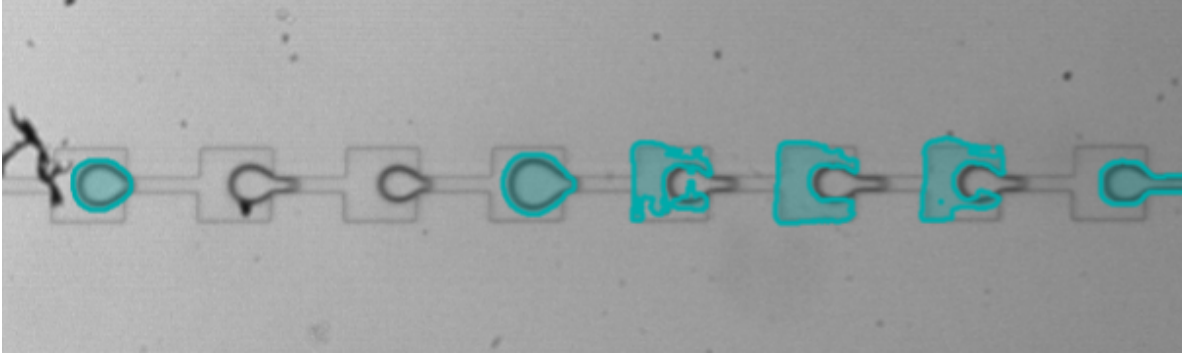


Figure 3.8: Mode B failure example where pre-annotations are significantly incorrect and require substantial manual correction.

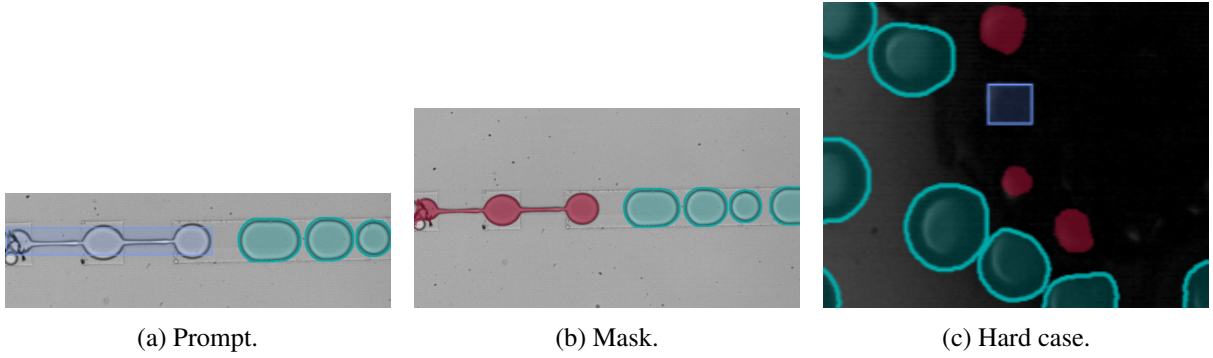


Figure 3.9: Mode C (SAM2-assisted refinement) in Label Studio: example interaction, resulting mask, and a challenging case requiring additional refinement.

reductions were computed between modes (manual $\rightarrow$ pre-annotations, and pre-annotations $\rightarrow$ pre-annotations+SAM2) to quantify relative gains.

3.6 Evaluation protocol

This section defines the evaluation procedure used to compare trained models on held-out test splits. The protocol is designed to (i) quantify foreground delineation quality at the pixel level, (ii) derive bubble-by-bubble statistics relevant to multiphase-flow measurements (count and area), and (iii) report indicative inference throughput under a consistent runtime setup. All evaluation runs operate on COCO-style instance segmentation datasets [18] and use the same deterministic preprocessing as training (resize and tensor conversion), with random augmentation disabled.

3.6.1 Validation-time model selection versus test-time reporting

Two complementary evaluation perspectives are used at different stages. During training/validation of Mask R-CNN models, checkpoint selection follows standard COCO instance segmentation evaluation,

using the segmentation average precision $AP@[0.50:0.95]$ [18, 5]. In contrast, the final held-out test evaluation reported in this dissertation uses a dedicated evaluator that computes (a) semantic overlap metrics from the union of predicted instance masks and (b) instance-derived bubble statistics (count and area) from filtered predictions. This test-time protocol is intentionally aligned with downstream measurement objectives, where stable occupancy/delineation and bubble count/size statistics are central.

3.6.2 Test dataloader, preprocessing, and reproducibility controls

The evaluator loads ground-truth masks from the test split `_annotations.coco.json` and decodes instance masks using standard COCO utilities [18]. A test dataloader is constructed with `batch_size=1` and `shuffle=False` to enable per-image reporting. The preprocessing pipeline is identical to training preprocessing (resize to the configured resolution, conversion to `float32` with scaling, and tensor conversion), while random augmentation is disabled to ensure deterministic evaluation.

To make qualitative examples reproducible, the evaluator uses a fixed `example_seed` (when provided) to select a consistent subset of test indices for visualization. The evaluation configuration used (thresholds and seed) is stored in the experiment summary output.

3.6.3 Prediction filtering and derived representations

For instance segmentation models (Mask R-CNN variants [12, 24]), each predicted instance is filtered using fixed thresholds defined in the YAML `eval` block: (i) confidence-score threshold `score_thresh`, (ii) mask-binarization threshold `mask_thresh` applied to the predicted mask probabilities, and (iii) minimum pixel area `min_area` to remove small spurious detections. These thresholds are kept identical across experiments to ensure fair comparisons.

After filtering, the evaluator constructs two complementary representations:

- an **instance set** (kept masks and boxes), used for bubble count and area statistics; and
- a **semantic foreground mask**, formed as the pixelwise union of all kept instance masks.

The same union operation is applied to the ground truth (union of all GT instance masks). Because semantic metrics are computed on the union mask, filtering thresholds influence both semantic and instance-level results. Moreover, union-mask metrics quantify foreground delineation/occupancy and do not fully penalize instance splitting or merging; therefore, semantic overlap is interpreted together with counting error and area statistics.

3.6.4 Metrics reported

Pixel-level (semantic) overlap metrics. From the merged prediction mask and merged ground-truth mask, the evaluator computes Precision, Recall, and F1-score on flattened binary masks, as well as Dice coefficient and Intersection over Union (IoU). If both the ground truth and prediction are empty for

a frame (no bubbles present and none predicted), all semantic metrics are set to 1.0 to reflect perfect agreement on the absence of bubbles.

Instance-derived bubble statistics. From the filtered instance set, the evaluator reports per-image bubble count (number of kept instances), per-image mean and median bubble area, and corresponding ground-truth values computed from GT masks. Bubble area is defined as the pixel count of the binarized instance mask, and is therefore reported in pixel units. In addition, the evaluator reports counting error ($\text{pred} - \text{gt}$) and absolute counting error per image, and flags frames where the ground truth is empty and/or the prediction is empty.

Inference-time indicators. Inference timing is measured per image using wall-clock time around the model forward pass. When running on GPU, CUDA synchronization is performed immediately before and after inference to ensure timings correspond to completed GPU execution. The evaluator reports average inference time per image and frames per second (FPS). Since no dedicated warm-up phase is used, these timing results are interpreted as indicative and are primarily used for relative comparisons under the same runtime environment.

3.6.5 Outputs and cross-experiment post-processing

For each experiment, the evaluator produces: (i) a per-image CSV file (`test_semantic_metrics.csv`) containing semantic overlap metrics, instance-derived statistics, and empty-frame flags; (ii) a structured JSON summary (`results.json`) containing averaged metrics, timing summaries, and the evaluation configuration used; and (iii) an `examples/` directory with qualitative visualizations (image, GT overlay, predicted mask overlay, and predicted boxes).

After evaluation, thesis-ready plots and comparison figures are generated from `results.json` and `test_semantic_metrics.csv`. The plotting module supports both single-experiment distributions (e.g., Dice/IoU histograms and bubble-count distributions) and multi-experiment bar-plot comparisons for key metrics and throughput, enabling standardized reporting across training regimes and architectures.

3.7 Implementation and reproducibility

This dissertation uses a configuration-driven experimental framework to ensure that training and evaluation runs are reproducible and directly comparable across model variants and training regimes.

3.7.1 Configuration-driven experiments

All experiments are parameterized through YAML configuration files. Each configuration specifies (i) an experiment identifier and random seed, (ii) the model variant and task definition (`num_classes`),

(iii) dataset paths for train/validation/test splits, (iv) optimization and training hyperparameters (batch size, learning rate, momentum, weight decay, scheduler), (v) optional augmentation switches, and (vi) fixed evaluation thresholds (score/mask thresholds and minimum area). Training is executed by passing the configuration path to the corresponding pipeline script (e.g., `-config <file.yaml>`), ensuring that the full run state is defined by the YAML file and the dataset split folders.

3.7.2 Training scripts, run structure, and checkpointing

Two training pipelines are used:

- `synthetic_train_pipeline.py` implements supervised training on the dataset splits defined in the YAML configuration.
- `transfer_train_pipeline.py` implements synthetic-to-real fine-tuning by initializing from a checkpoint (`transfer.init_checkpoint`) and training on real dataset splits specified in the YAML configuration.

Each run creates a time-stamped output directory under the base output folder defined in the YAML configuration (e.g., `outputs/<experiment>_<timestamp>/`). Per-epoch progress is recorded to a CSV metrics log, and the best-performing checkpoint is saved as `model_best.pth`. For Mask R-CNN runs, “best” is defined by the validation criterion used during training, which follows COCO-style instance segmentation evaluation (segmentation AP@[0.50:0.95]) [18, 5]. This validation-time criterion is used for checkpoint selection, while final test reporting follows the separate evaluation protocol described in Section 3.6.

3.7.3 Logging and configuration traceability

Training curves and scalar metrics are logged with TensorBoard when enabled in the YAML configuration. In addition, configuration text (including the full YAML content) is recorded in the run logs to preserve a complete trace of hyperparameters, thresholds, and dataset paths used for each experiment. This supports auditability and reduces the risk of undocumented configuration drift between runs.

3.7.4 Evaluation artifacts

Evaluation outputs (CSV/JSON summaries and qualitative overlays) are produced by the evaluator described in Section 3.6.

3.7.5 Random seeds and reproducibility scope

A fixed pseudo-random seed is specified per experiment via the YAML file and is applied to Python, NumPy, and PyTorch random number generators at the start of training (including CUDA seeds when

available). This reduces run-to-run variance and supports consistent comparisons. However, strict determinism is not enforced by default (e.g., some CUDA kernels may be non-deterministic), so exact bitwise reproducibility is not guaranteed across different hardware or execution settings. In this dissertation, comparisons are made under a consistent software stack and execution environment to mitigate this effect.

3.7.6 Execution environment

All experiments were executed on a personal workstation equipped with an NVIDIA GeForce RTX 2060 Max-Q GPU and 16 GB of system RAM. This hardware context motivates several practical configuration choices (batch size, image resolution, and the use of mixed precision) and provides a reference for interpreting training time and inference throughput reported later.

3.7.7 Software stack

All training and evaluation code is implemented in PyTorch and TorchVision, using TorchVision reference implementations of Mask R-CNN model variants and standard PyTorch components such as optimizers, learning-rate scheduling, and DataLoader abstractions [21, 5].

Chapter 4

Experimental Results and Discussion

4.1 Chapter overview and evaluation scope

This chapter reports the quantitative experimental results obtained with the unified PyTorch/TorchVision training and evaluation pipeline described in Chapter 3. The goal is to compare how (i) backbone capacity and (ii) training regime choices influence real-domain performance for bubble instance segmentation in multiphase experimental imagery, under a limited amount of labeled real data. Concretely, the evaluation grid consists of three TorchVision Mask R-CNN variants (`maskrcnn_resnet50_fpn`, `maskrcnn_resnet50_fpn_v2`, and `maskrcnn_resnet101_fpn`), each trained under four regimes: synthetic-only training, real-only baseline training, synthetic-to-real transfer learning (synthetic initialization followed by fine-tuning on real data), and transfer learning with augmentation enabled during fine-tuning. This yields a total of 12 experiments, which are compared using a fixed test protocol on the held-out real experimental test split organized in COCO-style instance segmentation format [18, 12, 24, 17, 21].

All quantitative comparisons in this chapter are computed on the same held-out experimental test split, with post-processing thresholds held fixed across runs. Specifically, predicted instances are filtered using: (i) a confidence-score threshold, (ii) a mask binarization threshold, and (iii) a minimum-area filter to remove small spurious detections. After filtering, two complementary families of indicators are computed. First, “semantic” overlap metrics are evaluated on the union (logical OR) of all predicted instance masks versus the union of all ground-truth instance masks. These include pixel-level Precision, Recall, F1-score, Dice, and IoU, and they quantify the accuracy of foreground delineation and occupancy. Second, measurement-oriented indicators are derived from the retained instances, including predicted bubble count, summary area statistics (mean and median area per image), and counting error relative to ground truth. Because union-mask overlap does not fully penalize split/merge errors between instances, overlap scores are interpreted together with counting error and area statistics to assess practical usefulness for downstream measurement (bubble counts and size distributions) in dense regimes [9, 13, 28, 32].

In addition to final test-set evaluation, training dynamics are summarized where relevant using the

experiment logs and TensorBoard curves. During training, scalar summaries of the loss components are recorded each epoch, and validation-time COCO segmentation AP (mAP) computed on the validation split is used for early stopping and for selecting the best checkpoint. TensorBoard therefore serves as a diagnostic tool to verify convergence behavior and detect overfitting across regimes, while the final comparisons reported in this chapter rely on the standardized test protocol and the same fixed evaluation thresholds for all experiments [18, 21].

The remainder of the chapter is organized as follows. Section 4.2 presents the main comparison table across all 12 experiments. Sections 4.3–4.4 isolate the effect of synthetic-to-real transfer learning and augmentation, respectively. Section 4.5 discusses backbone capacity versus inference throughput, and Section 4.6 analyzes stratified performance by scene difficulty (e.g., bubble density and scale). Finally, Section 4.7 provides qualitative error analysis using representative overlay examples, linking typical failure modes to the metric design and measurement objectives.

4.2 Summary of results and main comparison table

Table 4.1 summarizes the principal quantitative comparison across the 12 experiments (3 backbones $\times$ 4 training regimes) evaluated on the held-out real experimental test split (25 images). All runs follow the fixed post-processing and evaluation protocol defined in Section 3.6, using identical thresholds (score_thresh=0.5, mask_thresh=0.5, min_area=50) so that differences in performance are attributable to the model and training regime rather than per-run tuning [18]. Semantic overlap metrics (Precision, Recall, Dice, IoU) are computed on the union of predicted instance masks versus the union of ground-truth instance masks, while bubble-count indicators are derived from the filtered instance set [18]. Inference throughput (FPS) is measured per image with GPU synchronization around the forward pass (indicative timing) [21]; the results reported here were obtained on an NVIDIA RTX 2060 Max-Q (6 GB) with an Intel Core i5-10300H host CPU. Finally, the test split contains no empty frames (num_empty_gt=num_empty_pred=0 for all runs), so overlap metrics are not affected by the evaluator’s empty-frame convention [18].

Several consistent trends emerge from the main comparison. First, training directly on real data (Real baseline) yields high union-mask overlap across all backbones (Dice $\approx$ 0.92–0.96), confirming that Mask R-CNN can delineate bubbly foreground reliably under the controlled split and thresholds. Second, synthetic-only training exhibits a domain-gap effect that depends on the backbone: while ResNet-50-FPN achieves relatively strong union-mask overlap (Dice=0.892), it still shows larger counting errors ($|\Delta N| = 2.48$) than real-trained models, and the ResNet-101-FPN synthetic run performs substantially worse both in overlap (Dice=0.629) and counting stability ($|\Delta N| = 9.84$), indicating limited transferability from purely synthetic appearance in the most challenging setting [18].

Third, synthetic-to-real transfer learning improves measurement stability most strongly. For example, ResNet-101-FPN improves from Dice=0.921 and $|\Delta N| = 3.52$ (Real) to Dice=0.955 and $|\Delta N| = 0.96$ (Transfer), reducing systematic count bias while also increasing overlap. The best overall result in this

Table 4.1: Main comparison of the 12 experiments on the real experimental test split (25 images). All runs use the same post-processing thresholds (score_thresh=0.5, mask_thresh=0.5, min_area=50). Metrics are averaged per image. $|\Delta N|$ is the mean absolute bubble-count error and ΔN is the signed mean count error (Pred-GT). Inference timing (FPS) is measured on an NVIDIA RTX 2060 Max-Q 6GB (Intel Core i5-10300H host). No empty frames occur in this test split (#EmptyGT=#EmptyPred=0 in all runs).

Backbone	Regime	Dice $\uparrow$	IoU $\uparrow$	Prec. $\uparrow$	Rec. $\uparrow$	$ \Delta N $ $\downarrow$	ΔN	FPS $\uparrow$
maskrcnn_resnet50_fpn	Synthetic	0.892	0.824	0.891	0.902	2.48	2.24	5.23
maskrcnn_resnet50_fpn	Real	0.953	0.914	0.929	0.981	0.60	0.60	6.52
maskrcnn_resnet50_fpn	Transfer	0.954	0.916	0.927	0.986	0.44	0.44	7.01
maskrcnn_resnet50_fpn	Transfer+Aug	0.951	0.912	0.928	0.979	0.88	0.80	6.88
maskrcnn_resnet50_fpn_v2	Synthetic	0.740	0.657	0.702	0.906	2.48	1.28	4.27
maskrcnn_resnet50_fpn_v2	Real	0.960	0.925	0.950	0.971	0.48	0.32	5.45
maskrcnn_resnet50_fpn_v2	Transfer	0.963	0.930	0.950	0.977	0.40	0.16	6.05
maskrcnn_resnet50_fpn_v2	Transfer+Aug	0.962	0.928	0.950	0.974	0.48	0.24	5.19
maskrcnn_resnet101_fpn	Synthetic	0.629	0.509	0.887	0.535	9.84	-7.12	4.41
maskrcnn_resnet101_fpn	Real	0.921	0.866	0.911	0.944	3.52	3.52	3.42
maskrcnn_resnet101_fpn	Transfer	0.955	0.915	0.935	0.977	0.96	0.80	5.58
maskrcnn_resnet101_fpn	Transfer+Aug	0.941	0.893	0.945	0.940	0.96	0.96	5.69

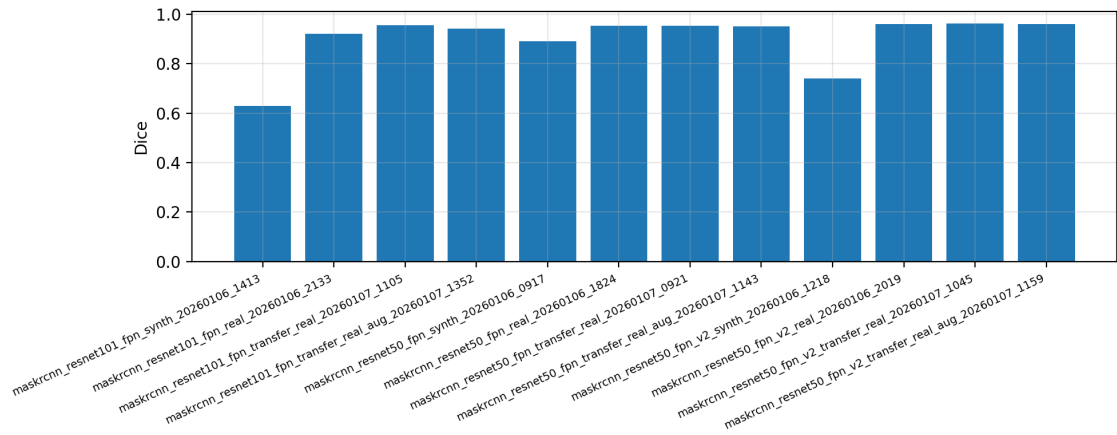
grid is obtained with the updated ResNet-50-FPNv2 backbone under transfer learning, which achieves the highest mean Dice (0.963) and IoU (0.930) together with the lowest mean absolute count error ($|\Delta N| = 0.40$). In terms of throughput, the fastest configuration is ResNet-50-FPN under transfer learning (7.01 FPS), providing the strongest speed/accuracy compromise among the evaluated models under the same runtime environment [21].

For readability, the full 12-experiment bar-plot comparisons (Dice, $|\Delta N|$, FPS) are used in this section as an overview of the grid. Subsequent sections present focused paired comparisons to isolate the effect of transfer learning (Real vs Transfer), augmentation (Transfer vs Transfer+Aug), and backbone capacity (R50 vs R50v2 vs R101) under the best-performing regime. Figure 4.1 provides a compact overview of accuracy, counting stability, and throughput across the full grid.

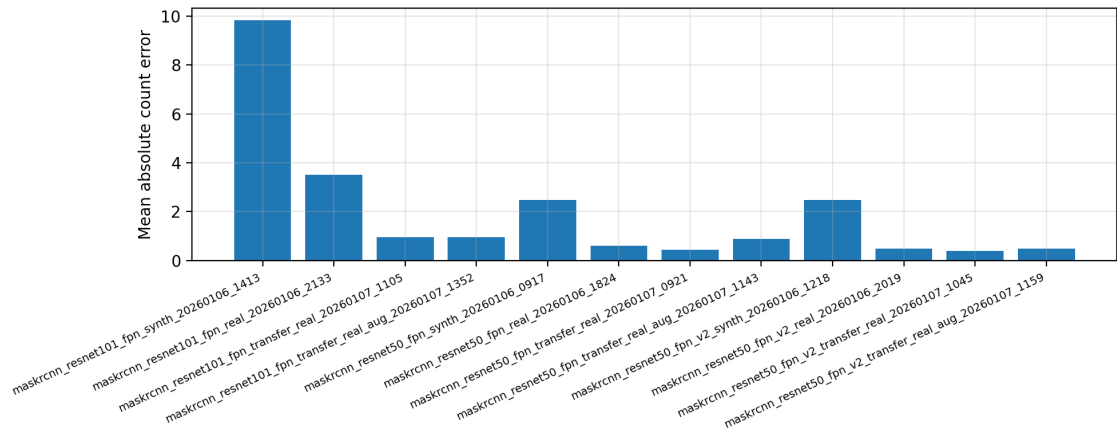
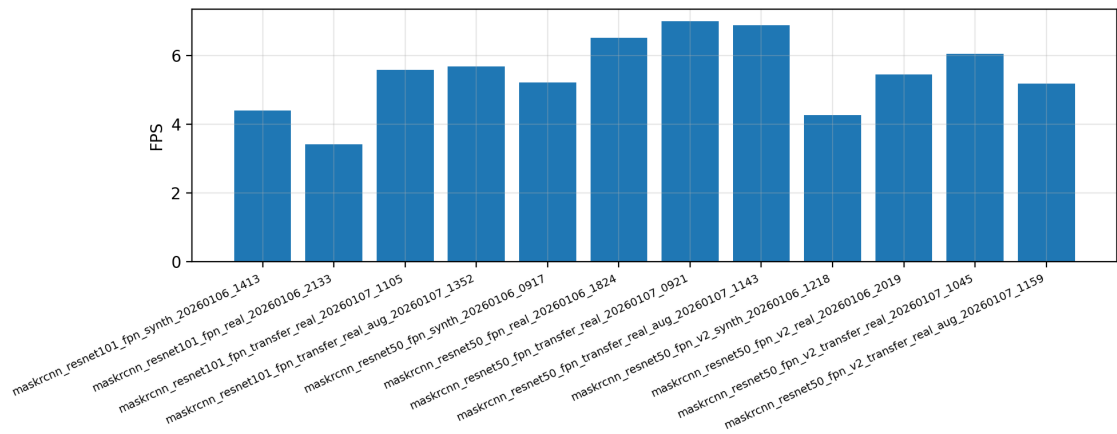
4.3 Effect of synthetic-to-real transfer learning

This section isolates the effect of synthetic-to-real transfer learning by comparing, for each backbone, a real-only baseline against a transfer configuration initialized from the corresponding synthetic-trained checkpoint and fine-tuned on the real training set. The motivation is that synthetic pretraining can provide a stronger initialization when labeled real data are limited, improving generalization after fine-tuning [20].

Test-set impact: overlap and measurement stability. Figures 4.2–4.4 summarize Real vs. Transfer performance on the held-out real test split. Table 4.2 reports the corresponding numerical deltas. For the ResNet-50 backbones, transfer learning yields small but consistent gains in union-mask overlap and



(a) Mean Dice (union-mask).

(b) Mean absolute count error $|\Delta N|$.

(c) Inference throughput (FPS).

Figure 4.1: Overview comparison across the 12 experiments (3 backbones $\times$ 4 regimes). All metrics are computed on the same real test split under fixed evaluation thresholds.

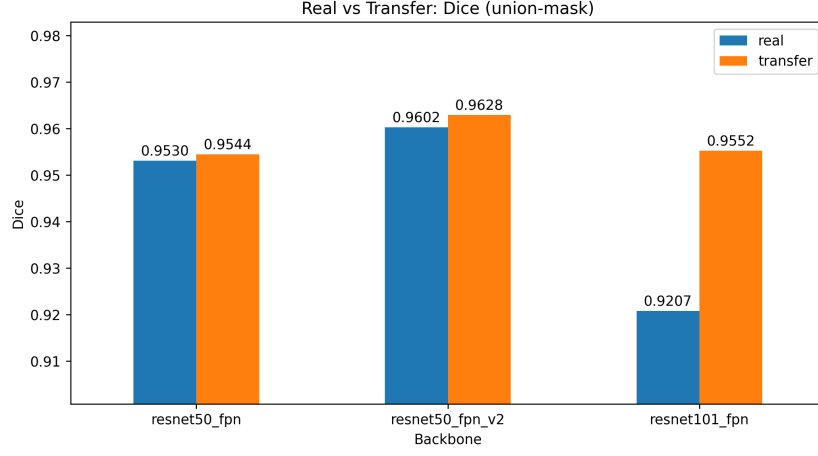


Figure 4.2: Effect of synthetic-to-real transfer learning on the real test split (Real vs. Transfer): mean Dice (union-mask) across backbones.

reduces counting errors: `resnet50_fpn` improves Dice from 0.9530 to 0.9544 while reducing the mean absolute count error $|\Delta N|$ from 0.60 to 0.44 bubbles/image; `resnet50_fpn_v2` improves Dice from 0.9602 to 0.9628 and reduces $|\Delta N|$ from 0.48 to 0.40. The strongest transfer effect is observed for `resnet101_fpn`, where Dice increases from 0.9207 to 0.9552 and $|\Delta N|$ decreases from 3.52 to 0.96, indicating a substantial improvement in both segmentation quality and instance-level counting stability.

To interpret systematic over/under-counting, the signed mean count error is defined as

$$\Delta N = E[N_{\text{pred}} - N_{\text{gt}}], \quad (4.1)$$

where $\Delta N > 0$ indicates systematic over-counting and $\Delta N < 0$ indicates under-counting. Transfer learning moves ΔN closer to zero for all backbones, most notably for `resnet101_fpn`, where the real-only baseline shows an average overcount of +3.52 and transfer reduces this bias to +0.80. Because union-mask overlap does not fully penalize instance split/merge errors, reductions in both $|\Delta N|$ and ΔN are particularly relevant for downstream measurement objectives such as bubble number density and size statistics [9, 13, 32].

Table 4.2: Real→Transfer deltas on the real test split (Transfer – Real). Positive deltas in Dice/IoU indicate improved overlap; negative deltas in $|\Delta N|$ indicate improved counting stability.

Backbone	ΔDice	ΔIoU	$\Delta \Delta N $	$\Delta(\Delta N)$
resnet50_fpn	+0.0014	+0.0024	−0.16	−0.16
resnet50_fpn_v2	+0.0026	+0.0046	−0.08	−0.16
resnet101_fpn	+0.0344	+0.0491	−2.56	−2.72

Training dynamics: validation mAP evidence (TensorBoard). The training pipeline logs COCO-style validation segmentation performance each epoch using mAP (AP@[0.50:0.95]) [18]. Figures 4.5–

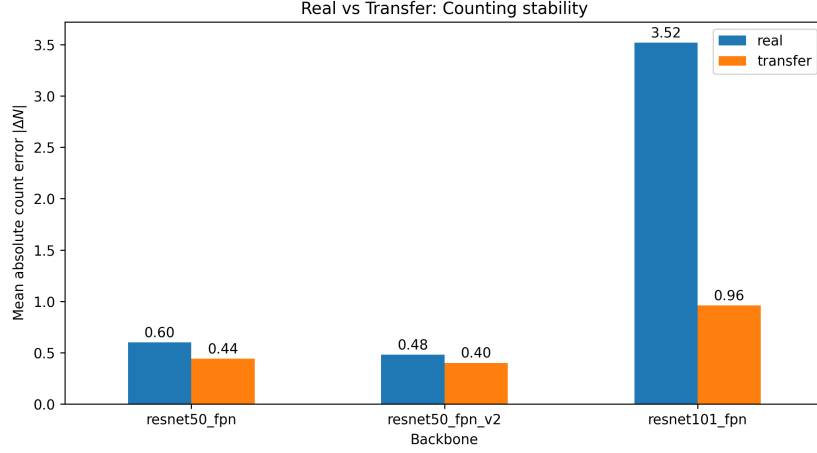


Figure 4.3: Effect of synthetic-to-real transfer learning on the real test split (Real vs. Transfer): mean absolute count error $|\Delta N|$ across backbones.

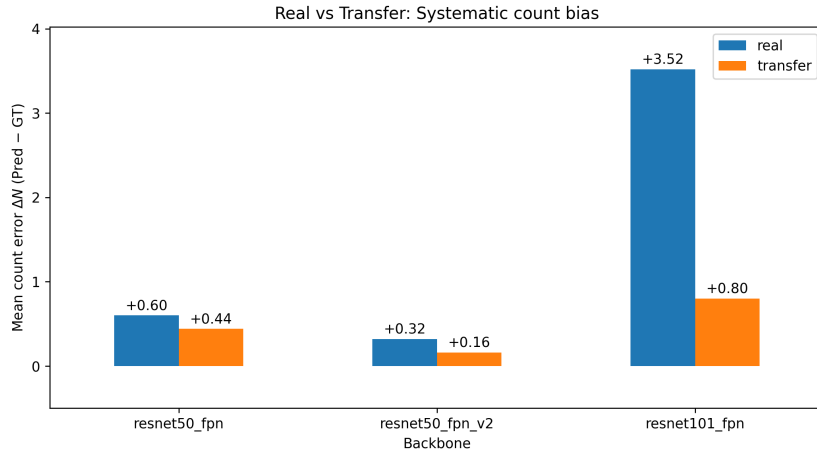


Figure 4.4: Effect of synthetic-to-real transfer learning on the real test split (Real vs. Transfer): mean signed count error ΔN (Pred – GT) across backbones.

4.7 show the TensorBoard validation curves for Real vs. Transfer runs. For `resnet50_fpn` and `resnet50_fpn_v2`, transfer learning reaches a slightly higher validation plateau; for `resnet101_fpn`, transfer learning produces a markedly higher validation trajectory and final plateau. The best validation mAP values extracted from the logged training CSVs (Table 4.3) corroborate this observation: transfer increases best validation mAP from 0.8871 to 0.8979 (`resnet50_fpn`), from 0.8867 to 0.8955 (`resnet50_fpn_v2`), and from 0.8298 to 0.8657 (`resnet101_fpn`). Overall, the TensorBoard results support the test-set findings: synthetic initialization improves optimization and generalization during fine-tuning, with the largest benefit for the higher-capacity backbone [20].

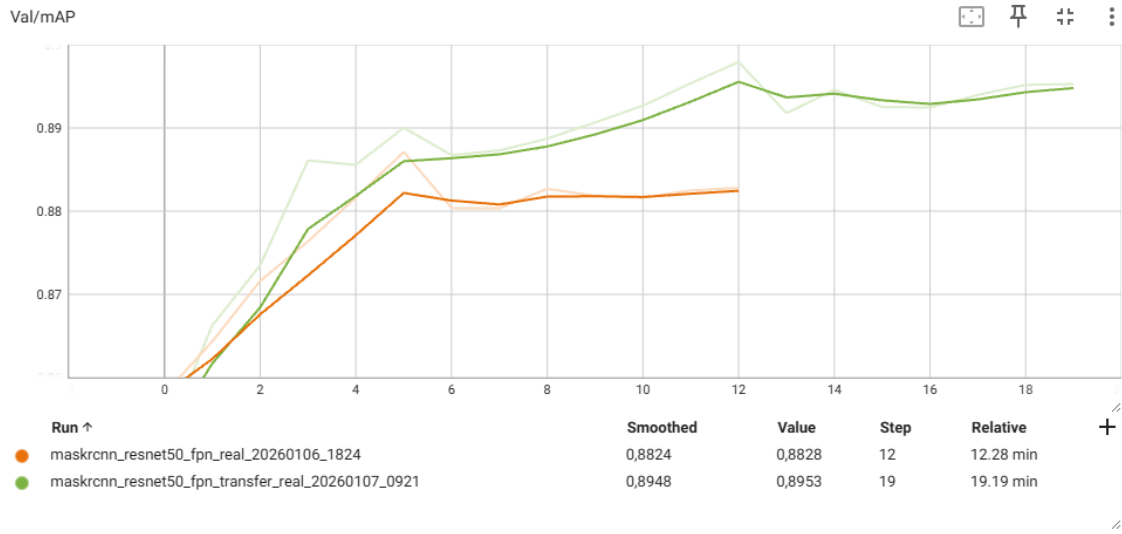


Figure 4.5: TensorBoard validation mAP (AP@[0.50:0.95]) versus epoch for resnet50_fpn: Real vs. Transfer training.

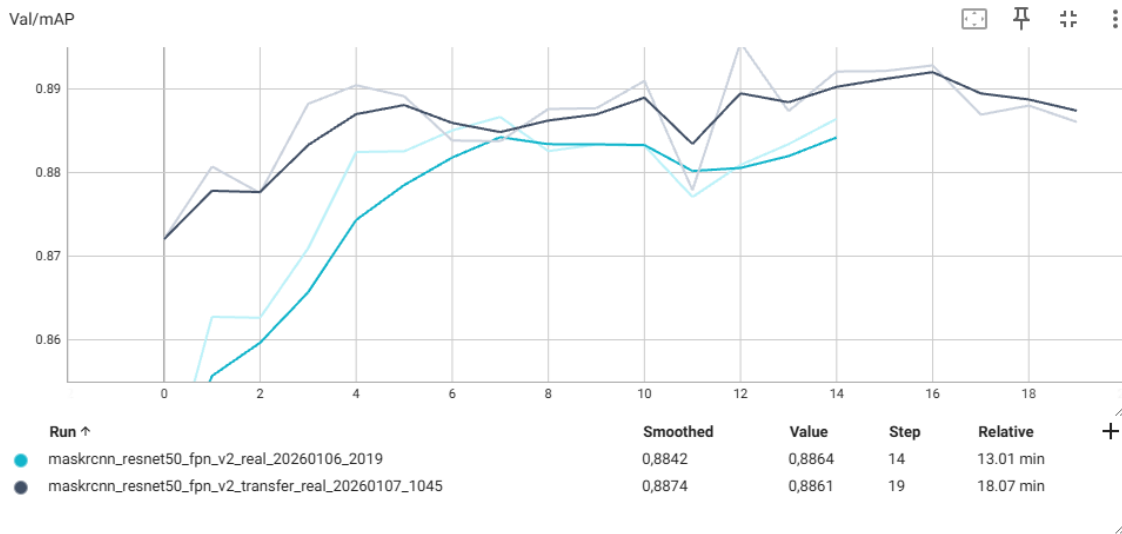


Figure 4.6: TensorBoard validation mAP (AP@[0.50:0.95]) versus epoch for resnet50_fpn_v2: Real vs. Transfer training.

4.4 Effect of augmentation during transfer learning

This section evaluates whether enabling data augmentation during the real fine-tuning stage improves the synthetic-to-real transfer regime. In principle, augmentation can increase robustness to nuisance variability (e.g., contrast changes, small geometric perturbations) and reduce overfitting when the labeled real dataset is limited [27]. Here, the comparison is performed by holding the backbone, data split, and evaluation thresholds fixed, and comparing *Transfer* versus *Transfer+Aug* for each of the three

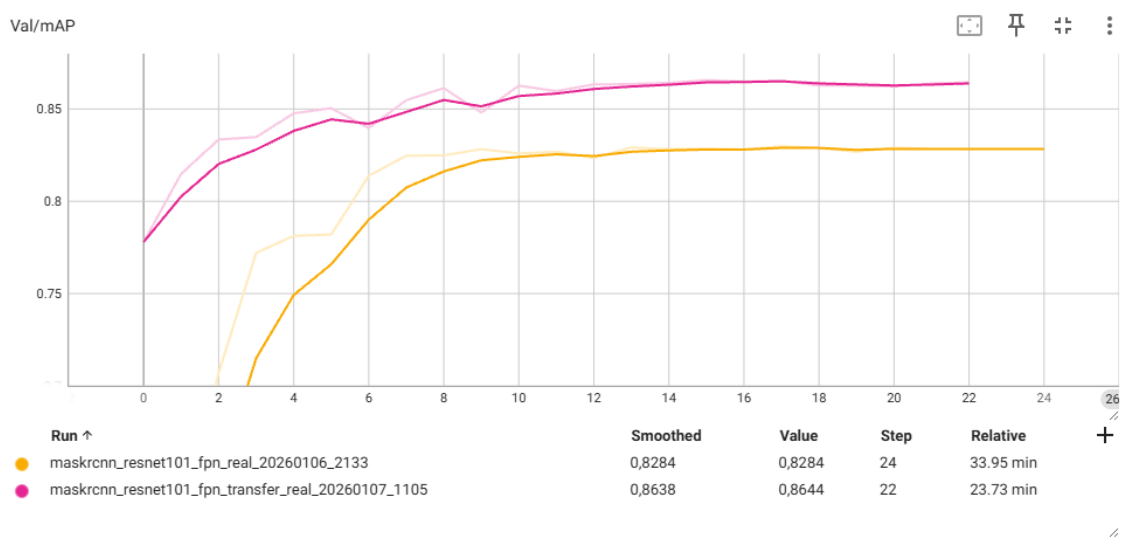


Figure 4.7: TensorBoard validation mAP (AP@[0.50:0.95]) versus epoch for resnet101_fpn: Real vs. Transfer training.

backbones.

Test-set impact: mean overlap and counting indicators. Figures 4.8–4.10 summarize the effect of augmentation on the held-out real test split. In contrast to transfer learning alone (Section 4.3), augmentation does not improve the mean semantic overlap metrics in this dataset. For resnet50_fpn, Dice decreases slightly (0.9544→0.9514) and counting stability worsens, with mean absolute count error increasing (0.44→0.88 bubbles/image) and the signed bias increasing (ΔN : 0.44→0.80). The same trend is observed for resnet50_fpn_v2, where Dice decreases marginally (0.9628→0.9616) and $|\Delta N|$ increases (0.40→0.48). For resnet101_fpn, Dice decreases more noticeably (0.9552→0.9413), while the mean absolute count error remains unchanged (0.96→0.96) and the signed count bias increases slightly (0.80→0.96). Overall, on this test split, augmentation does not yield a mean-performance gain and may introduce a small degradation in overlap and/or instance counting.

Robustness and error tails. Mean values can hide whether augmentation helps in the most difficult frames. To complement the mean metrics, the empirical cumulative distribution function (ECDF) of the per-image absolute count error $|\Delta N|$ is plotted. This curve reports, for each threshold t , the fraction of test images with $|\Delta N| \leq t$ and highlights tail behavior (rare large errors) [25]. To assess robustness, the distributions of $|\Delta N|$ for Transfer and Transfer+Aug are compared using the ECDF curves in Figures 4.11–4.13. For resnet101_fpn, augmentation reduces extreme errors even though the mean $|\Delta N|$ is unchanged: the upper-tail of the distribution contracts (e.g., the near-maximum observed error decreases from approximately 4.8 to 3.0 bubbles/image), and the fraction of frames with $|\Delta N| \geq 2$

Table 4.3: Best validation segmentation mAP (AP@[0.50:0.95]) recorded during training (from the per-epoch metrics CSV exported by the training pipeline).

Backbone	Real best mAP (epoch)	Transfer best mAP (epoch)	Δ best mAP
resnet50_fpn	0.8871 (5)	0.8979 (12)	+0.0108
resnet50_fpn_v2	0.8867 (7)	0.8955 (12)	+0.0089
resnet101_fpn	0.8298 (17)	0.8657 (15)	+0.0358

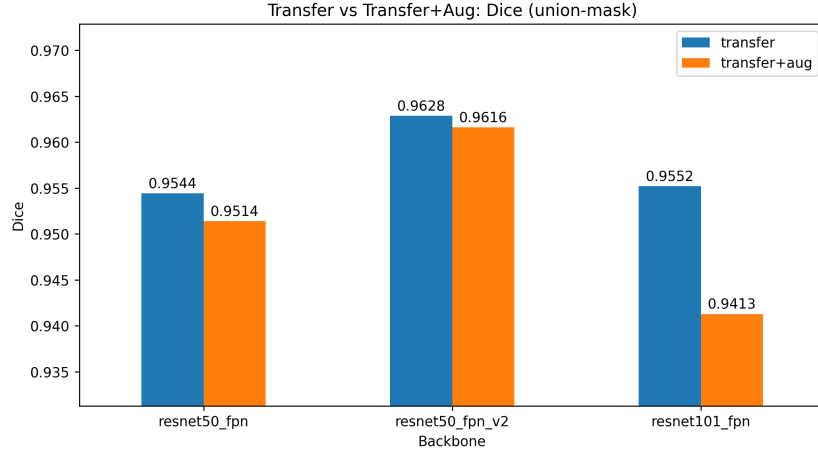


Figure 4.8: Effect of augmentation during transfer learning (Transfer vs. Transfer+Aug): mean Dice (union-mask) across backbones.

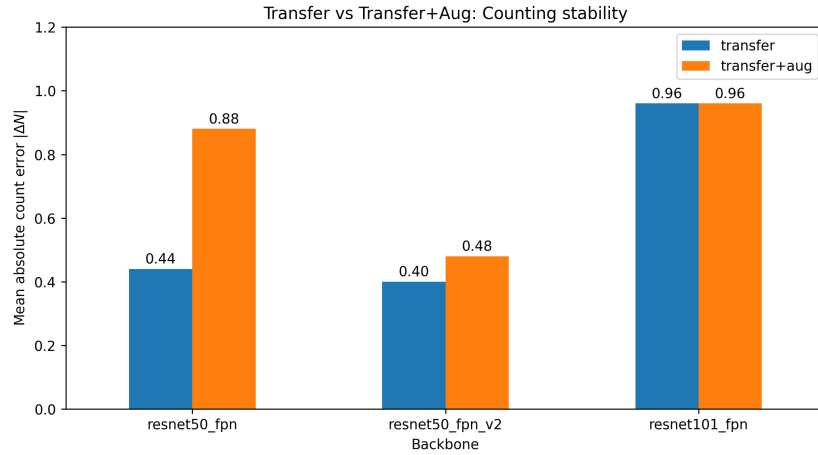


Figure 4.9: Effect of augmentation during transfer learning (Transfer vs. Transfer+Aug): mean absolute count error $|\Delta N|$ across backbones.

decreases (0.28→0.24). In contrast, for the ResNet-50 backbones the tail behavior worsens: the fraction of frames with $|\Delta N| \geq 2$ increases from 0.08 to 0.20 (*resnet50_fpn*) and from 0.08 to 0.12 (*resnet50_fpn_v2*). Given the relatively small test set (25 images), these tail statistics should be interpreted as indicative rather than definitive, but they suggest that the current augmentation setup is

Table 4.4: Transfer→Transfer+Aug changes on the real test split (Transfer+Aug – Transfer). Negative deltas in Dice/IoU indicate reduced overlap; positive deltas in $|\Delta N|$ indicate worse counting stability.

Backbone	ΔDice	ΔIoU	$\Delta \Delta N $	$\Delta(\Delta N)$
resnet50_fpn	−0.0030	−0.0047	+0.44	+0.36
resnet50_fpn_v2	−0.0012	−0.0022	+0.08	+0.08
resnet101_fpn	−0.0139	−0.0216	+0.00	+0.16

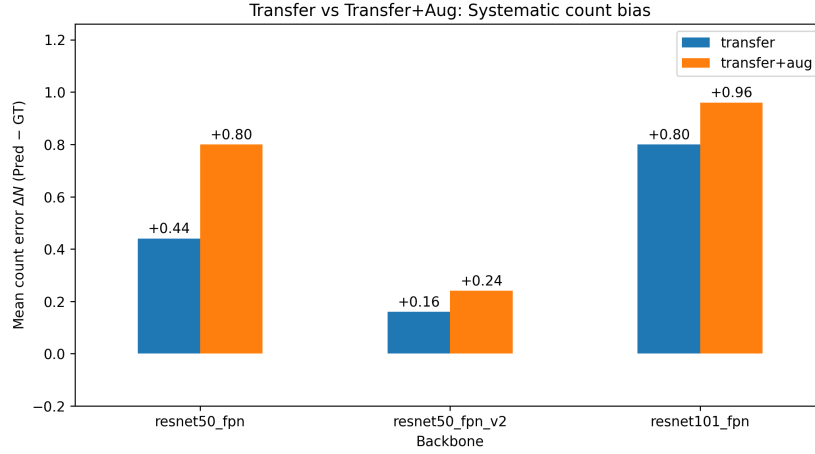


Figure 4.10: Effect of augmentation during transfer learning (Transfer vs. Transfer+Aug): mean signed count error ΔN (Pred – GT) across backbones.

not uniformly beneficial and may be more helpful for the higher-capacity model in the hardest cases [27].

Interpretation and limitations. The mixed or negative effect of augmentation in this study should be interpreted in the context of dataset scale and distribution. The real dataset available for fine-tuning is limited, and the held-out test split contains only 25 images. With such a small test set, performance estimates (both means and tail statistics) have high variance, and a small number of challenging frames can disproportionately influence the measured impact of augmentation. Moreover, augmentation improves robustness primarily when the applied transformations approximate plausible variability of the target acquisition conditions. If the augmentation policy is even partially mismatched to the real imaging process (e.g., overly strong photometric perturbations or geometric transformations that do not occur in the experimental setup), it can act as a form of distribution shift during training and degrade performance under the fixed test protocol. These observations suggest that (i) augmentation policy selection should be guided by knowledge of the imaging process and validated on a sufficiently large validation/test set, and (ii) the potential robustness benefits of augmentation are more reliably assessed when the evaluation split covers a broader range of operating conditions [27].

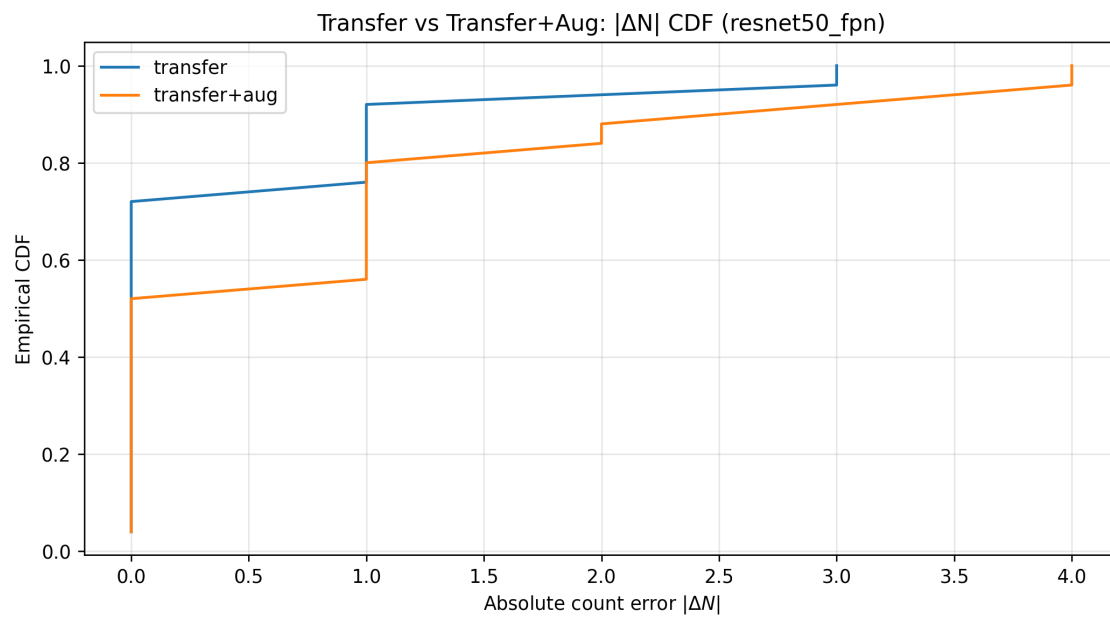


Figure 4.11: Empirical CDF of absolute count error $|\Delta N|$ for resnet50_fpn (Transfer vs. Transfer+Aug).

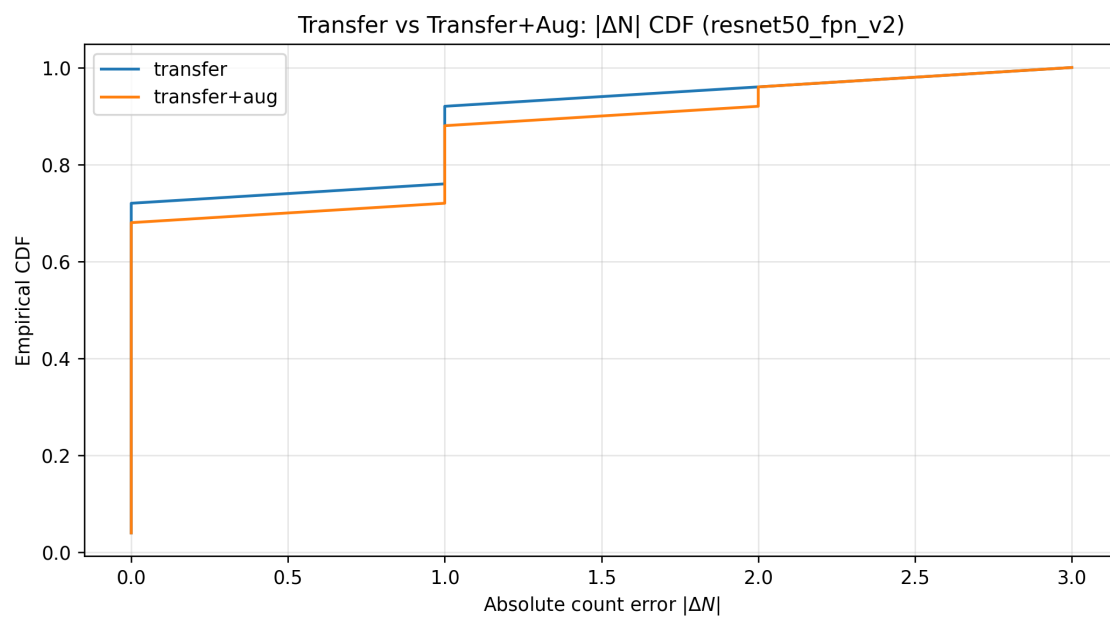


Figure 4.12: Empirical CDF of absolute count error $|\Delta N|$ for resnet50_fpn_v2 (Transfer vs. Transfer+Aug).

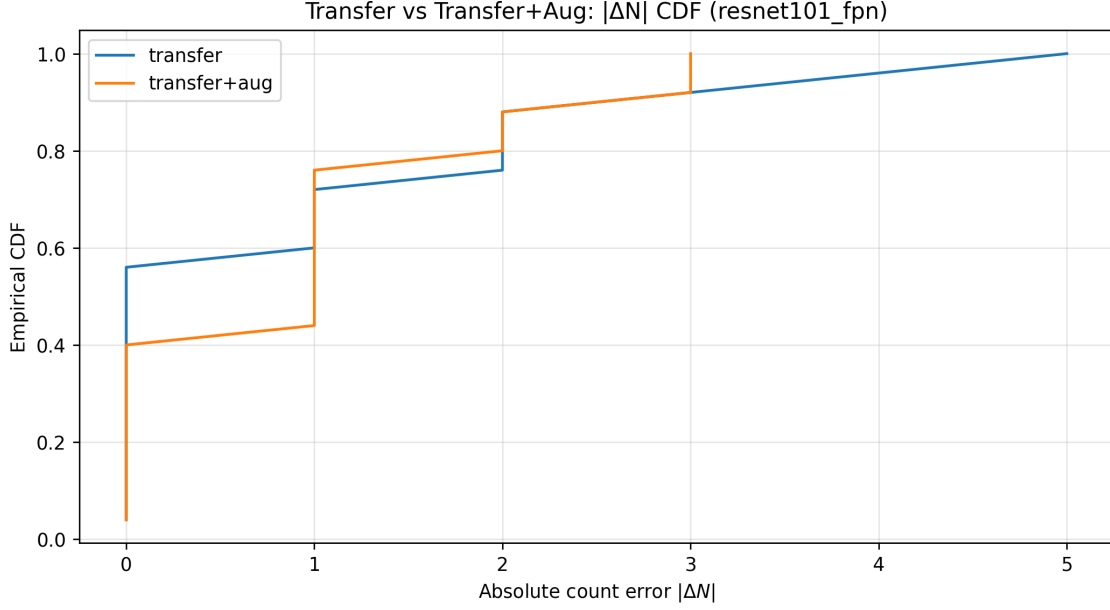


Figure 4.13: Empirical CDF of absolute count error $|\Delta N|$ for `resnet101_fpn` (Transfer vs. Transfer+Aug).

4.5 Backbone capacity vs inference speed

This section analyzes the trade-off between backbone capacity and inference throughput for the Mask R-CNN architectures evaluated in this work. While higher-capacity backbones can improve representation power, they typically increase computational cost (deeper feature extraction and higher memory bandwidth demands), which can reduce inference speed. This trade-off is particularly relevant for practical use cases where bubble segmentation is a preprocessing step for measurement and the processing rate may be constrained by hardware [12, 24, 17].

To provide a fair comparison, the analysis focuses on the best-performing training regime identified in the previous sections, namely synthetic-to-real transfer learning without additional augmentation. All models are evaluated on the same held-out real test split using identical post-processing thresholds. Inference timing is measured under the same runtime environment (NVIDIA RTX 2060 Max-Q 6GB, Intel Core i5-10300H @ 2.50 GHz), and the reported FPS should be interpreted as indicative throughput for the implemented pipeline (including model forward pass and post-processing) under these conditions.

Table 4.5: Backbone capacity versus inference throughput in the transfer regime (real test).

Backbone	Dice	IoU	$ \Delta N $	ΔN	FPS
resnet50_fpn	0.9544	0.9164	0.44	+0.44	7.01
resnet50_fpn_v2	0.9628	0.9298	0.40	+0.16	6.05
resnet101_fpn	0.9552	0.9151	0.96	+0.80	5.58

Table 4.5 shows that `resnet50_fpn` is the fastest configuration, achieving 7.01 FPS, while `resnet101_fpn`

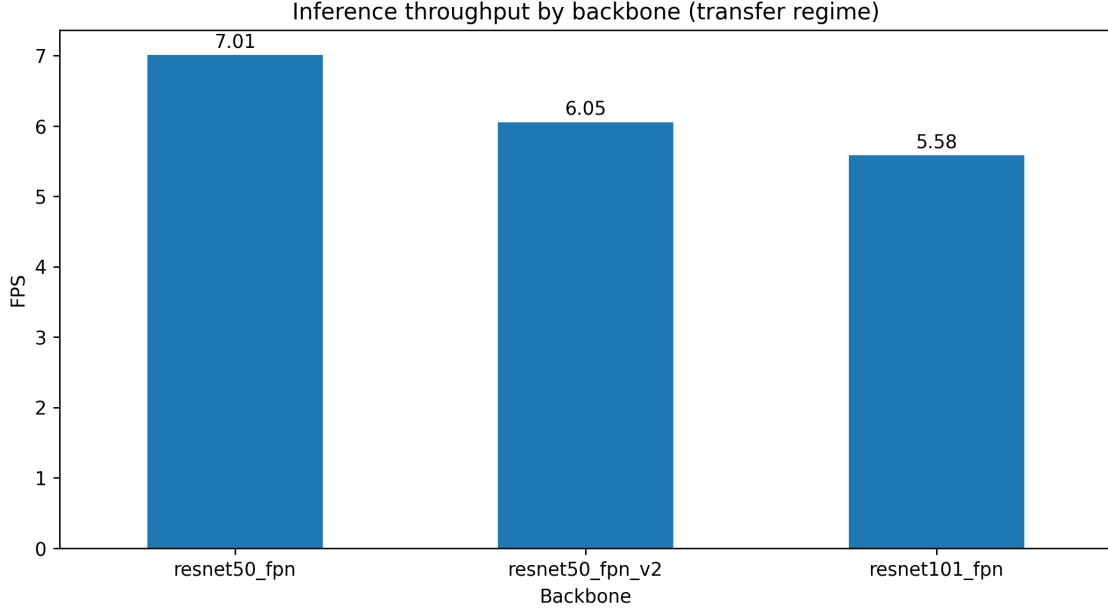


Figure 4.14: Inference throughput (FPS) by backbone in the transfer regime on the real test split (same hardware and evaluation pipeline).

is the slowest at 5.58 FPS. The updated `resnet50_fpn_v2` backbone provides an intermediate throughput of 6.05 FPS, corresponding to an approximately 14% slowdown relative to `resnet50_fpn`. In terms of accuracy and measurement stability, `resnet50_fpn_v2` provides the best overall test-set performance among the evaluated backbones in this regime (Dice=0.9628, $|\Delta N|=0.40$), while `resnet101_fpn` does not yield a corresponding improvement despite its higher capacity (Dice=0.9552, $|\Delta N|=0.96$). These results suggest that, for the available training set size and imaging conditions, increasing backbone depth from ResNet-50 to ResNet-101 increases computational cost without translating into better test-set performance.

Figures 4.14–4.16 visualize this trade-off. Figure 4.14 summarizes the throughput differences across backbones, while Figure 4.15 shows that `resnet50_fpn_v2` achieves the highest overlap metric with only a modest speed penalty. The speed–accuracy scatter in Figure 4.16 highlights `resnet50_fpn_v2` as the best compromise point for this dataset, whereas `resnet50_fpn` remains preferable if maximum throughput is the dominant constraint.

4.6 Stratified performance by scene difficulty

Although strongly overlapping bubbles are limited in the available real dataset, scene difficulty in multiphase-flow imagery is also driven by bubble density, bubble scale, and crowding effects that impact instance separation and counting stability. To analyze where errors concentrate, performance is stratified on the real test split using two ground-truth difficulty proxies computed per image: (i) bubble density,

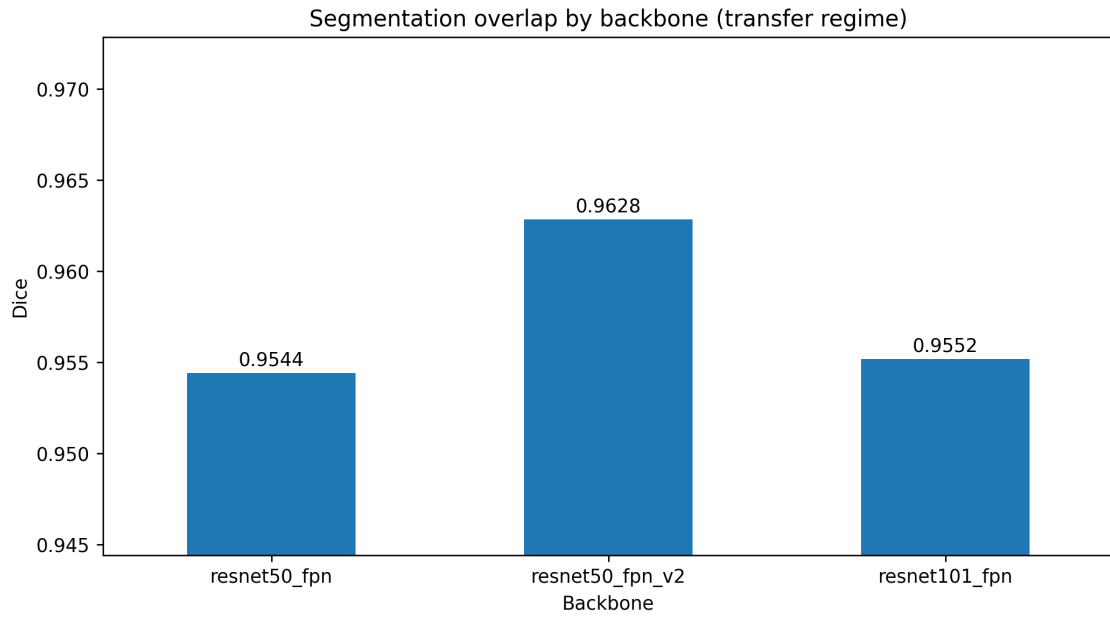


Figure 4.15: Mean Dice (union-mask) by backbone in the transfer regime on the real test split.

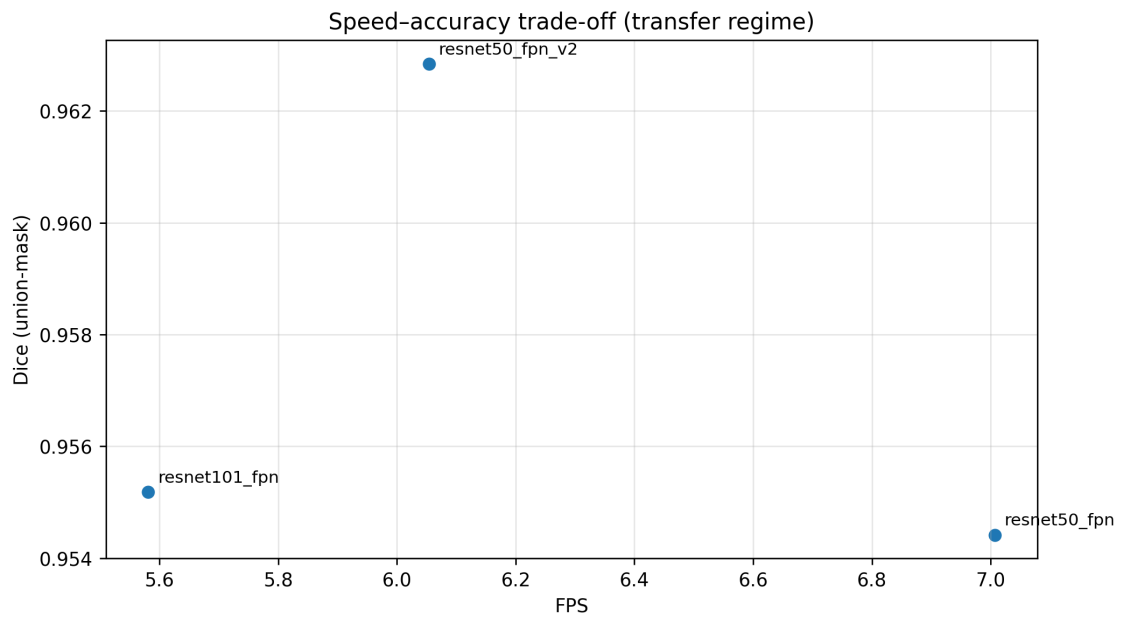


Figure 4.16: Speed-accuracy trade-off in the transfer regime (Dice versus FPS).

measured by GT_NumBubbles, and (ii) characteristic bubble scale, measured by GT_MeanArea. These quantities are available in the per-image evaluation logs and are independent of the prediction model.

Three density bins are defined from the test-set distribution (25 images): Low (1–8 bubbles), Mid (9–16), and High (17–48). Similarly, size bins are defined as Small, Medium, and Large based on the

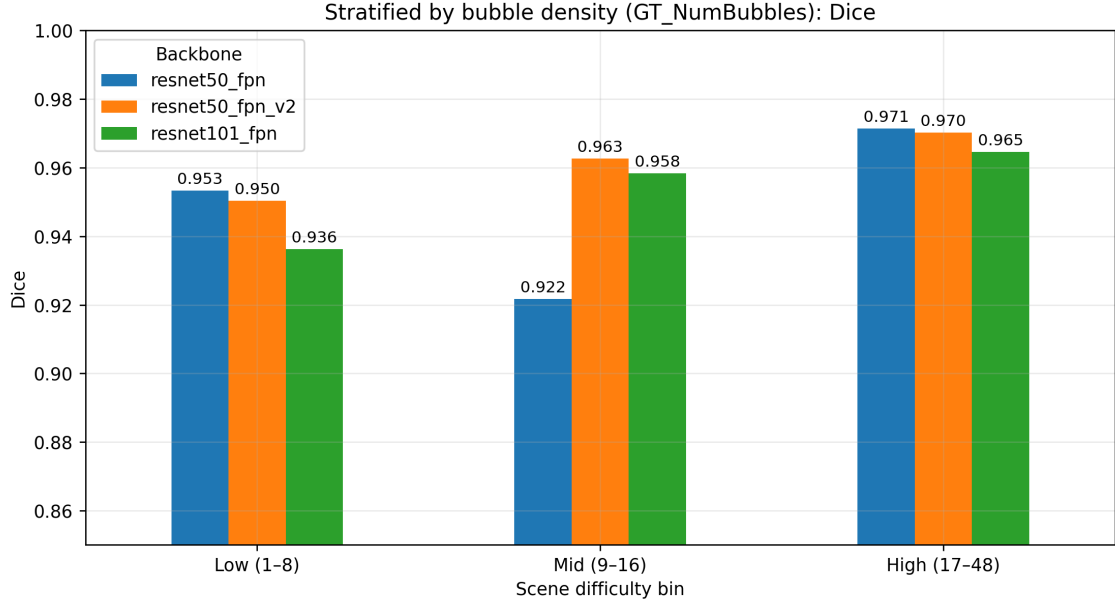


Figure 4.17: Stratified Dice (union-mask) in the transfer regime by bubble density (GT_NumBubbles bins).

GT_MeanArea quartiles. All stratified results in this section correspond to the synthetic-to-real transfer regime, as it provided the most consistent overall accuracy in previous comparisons.

Stratification by bubble density. Table 4.6 and Figures 4.17–4.19 show performance as a function of GT bubble density. For overlap quality, Dice remains high across bins for all backbones, with the highest Dice values generally observed in the high-density bin. Counting behavior, however, varies more noticeably by backbone: the ResNet-101 backbone shows substantially larger $|\Delta N|$ and positive ΔN in the high-density bin, indicating persistent over-counting in crowded scenes even when union-mask overlap is strong. In contrast, the ResNet-50 backbones exhibit lower mean absolute count error in the mid/high bins, suggesting more stable instance separation under the fixed post-processing thresholds.

Table 4.6: Stratified performance in the transfer regime by bubble density (bins defined by GT_NumBubbles quartiles on the 25-image real test split).

Bin	n	R50 Dice	R50 $ \Delta N $	R50v2 Dice	R50v2 $ \Delta N $	R101 Dice
R101 ΔN						
Low (1–8)	7	0.953	1.00	0.950	0.43	0.936
1.00						
Mid (9–16)	6	0.922	0.17	0.963	0.33	0.958
0.50						
High (17–48)	12	0.971	0.25	0.970	0.42	0.965
1.17						

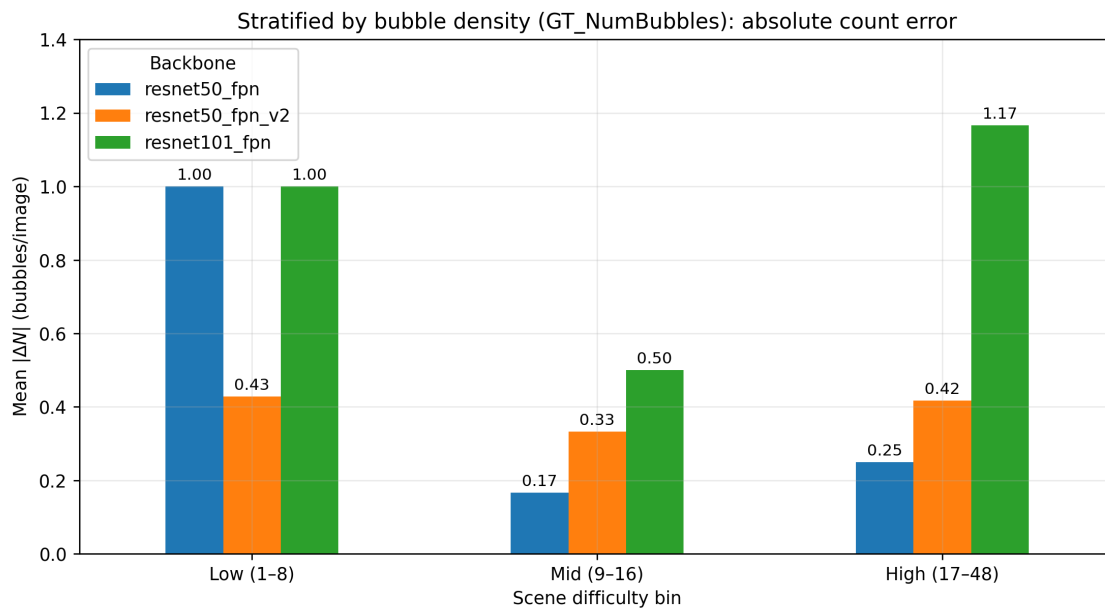


Figure 4.18: Stratified mean absolute count error $|\Delta N|$ in the transfer regime by bubble density (GT_NumBubbles bins).

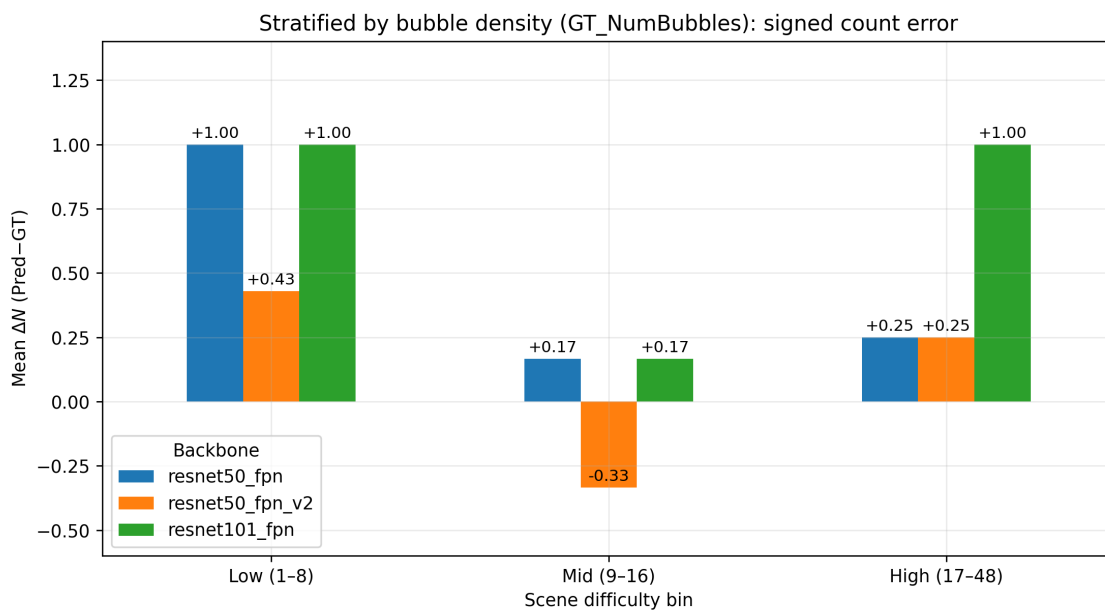


Figure 4.19: Stratified mean signed count error ΔN (Pred-GT) in the transfer regime by bubble density (GT_NumBubbles bins).

Stratification by bubble scale. Table 4.7 and Figures 4.20–4.21 show performance as a function of GT mean bubble area. A consistent pattern is that small-bubble frames tend to have lower Dice than medium/large frames for some backbones, which is expected since small instances occupy fewer pixels

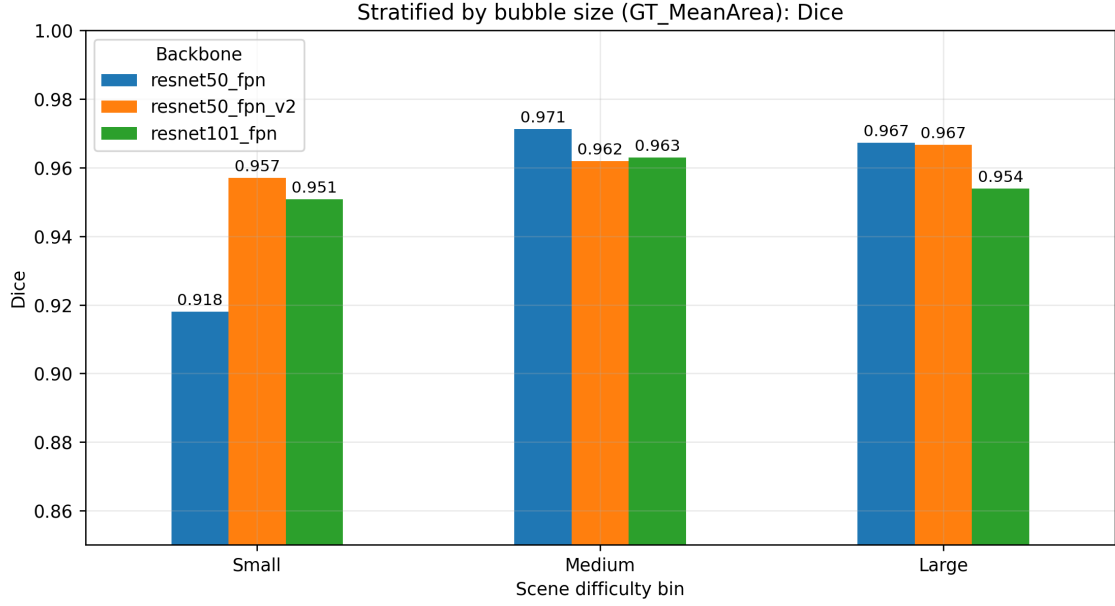


Figure 4.20: Stratified Dice (union-mask) in the transfer regime by bubble scale (GT_MeanArea bins).

and are more sensitive to thresholding and boundary errors. Counting stability also degrades for large-bubble frames in the ResNet-101 backbone, reflected by an increase in $|\Delta N|$ and ΔN , suggesting that the model tends to generate extra instances in these scenes (over-segmentation) under the fixed filtering parameters. Overall, the stratified analysis supports the earlier conclusion that union-mask overlap alone can appear strong even when instance-level measurement indicators degrade, reinforcing the need to report both overlap and instance-derived statistics.

Table 4.7: Stratified performance in the transfer regime by mean bubble size (bins defined by GT_MeanArea quartiles on the 25-image real test split).

Bin	n	R50 Dice	R50 $ \Delta N $	R50v2 Dice	R50v2 $ \Delta N $	R101 Dice
R101 ΔN						
Small	7	0.918	0.43	0.957	0.29	0.951
0.57						
Medium	6	0.971	0.17	0.962	0.67	0.963
0.83						
Large	12	0.967	0.58	0.967	0.33	0.954
1.25						

4.7 Qualitative error analysis

Quantitative metrics summarize average behavior but do not always reveal the visual causes of failure. This section complements the numerical comparisons by inspecting representative frames from the real

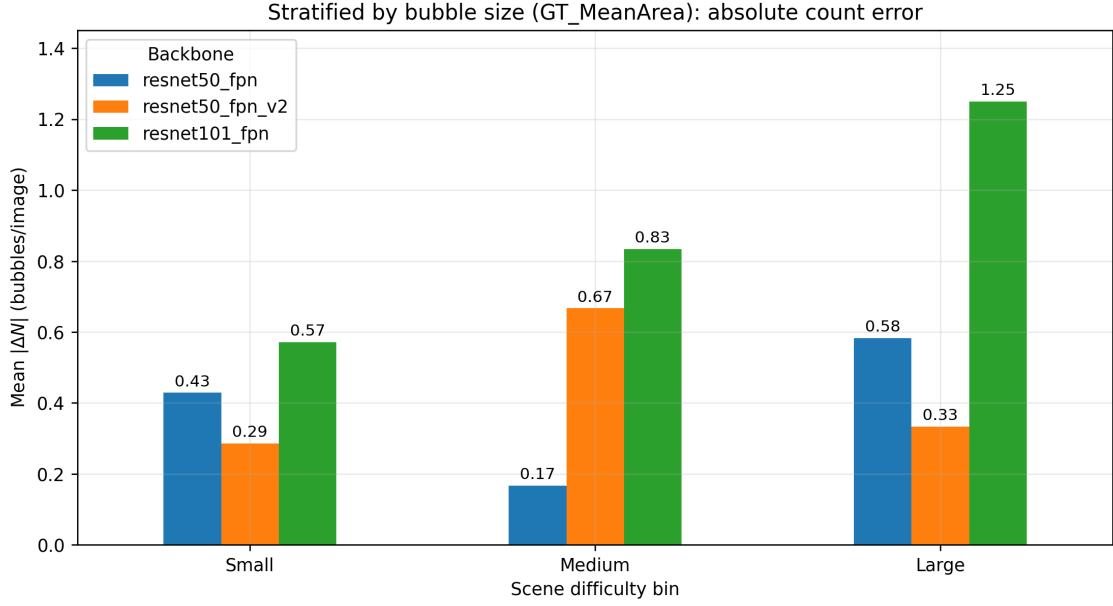


Figure 4.21: Stratified mean absolute count error $|\Delta N|$ in the transfer regime by bubble scale (GT_MeanArea bins).

test split that expose recurring error modes relevant to measurement. In particular, union-mask overlap (Dice/IoU) can remain high even when instance-level behavior degrades (missed bubbles, spurious detections, or split/merge effects), so qualitative inspection is used to contextualize the count-based indicators $|\Delta N|$ and ΔN reported in the previous sections.

Each example figure shows (left-to-right) the raw image, the ground-truth union mask overlay, and the predicted instance masks with bounding boxes. The selected cases are indexed by the test-set image order (ImageIdx) and were chosen to illustrate: (i) a typical success case with a backbone-dependent counting failure, (ii) false positives induced by bubble-like artifacts, (iii) dark and dense conditions that tend to increase overprediction, and (iv) a small/irregular-bubble scene that is handled well by the best-performing regime.

Case A: strong segmentation with backbone-dependent overcounting (ImageIdx=12). ImageIdx=12 is a representative frame where `resnet50_fpn_v2` produces visually consistent results in both the real-only and transfer regimes. However, `resnet101_fpn` in the transfer regime exhibits overcounting, consistent with the larger positive ΔN observed for the higher-capacity backbone in the quantitative results. This example illustrates an important point for downstream measurements: strong overlap can coexist with unstable instance counts, and increasing backbone capacity does not necessarily improve counting reliability for this dataset.

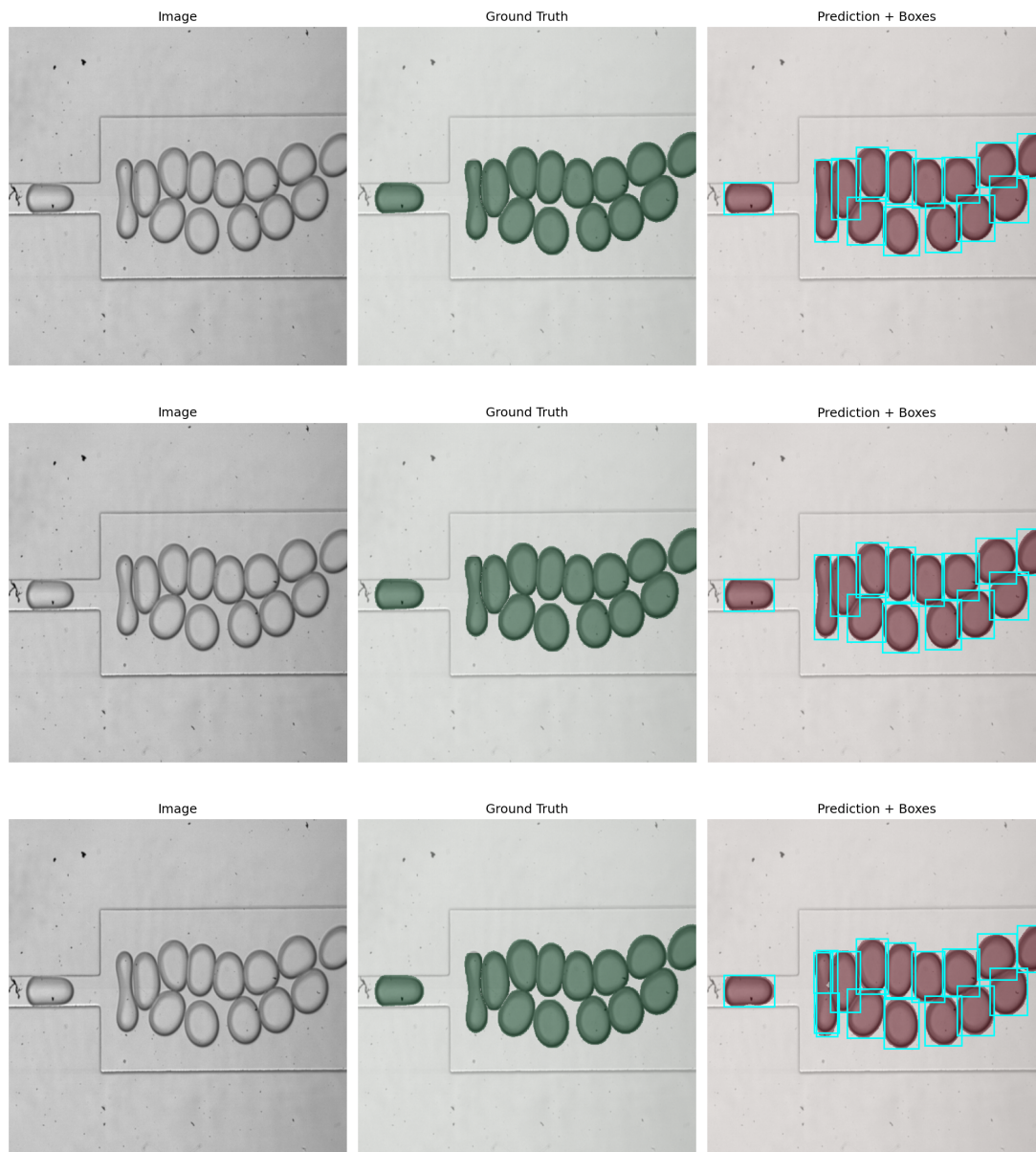


Figure 4.22: Qualitative comparison for ImageIdx=12. Top: resnet50_fpn_v2 real-only. Middle: resnet50_fpn_v2 transfer. Bottom: resnet101_fpn transfer, showing overcounting despite visually plausible overlap.

Case B: false positives induced by bubble-like structures (ImageIdx=5). ImageIdx=5 contains structures that resemble bubbles but are not annotated as such, leading to spurious detections. This is a challenging failure mode because false positives can appear visually reasonable and may not be strongly penalized by union-mask overlap when they occur outside the main bubble regions. The qualitative comparison shows that even the best-performing configuration can be affected by this ambiguity, while the higher-capacity backbone may exacerbate over-detection under the same thresholds.

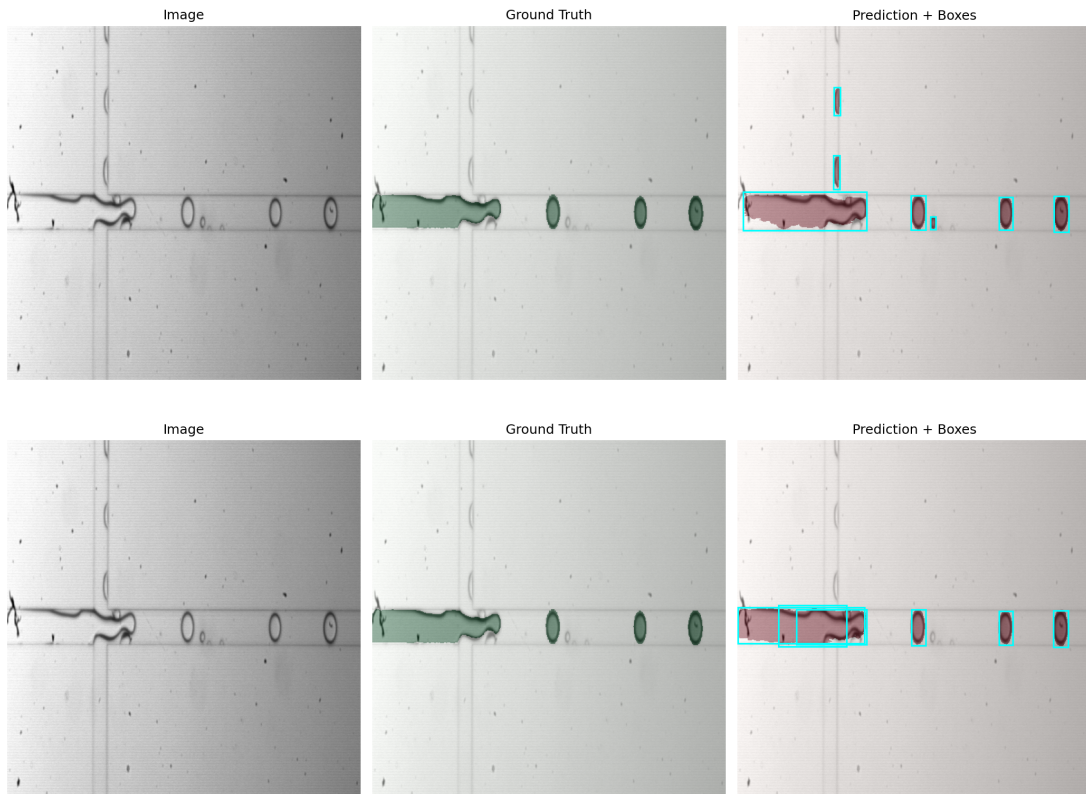


Figure 4.23: False-positive example for ImageIdx=5. Both models are challenged by bubble-like structures that are not true bubbles, leading to spurious instances and increased counting error.

Case C: dark and dense conditions with overprediction tendency (ImageIdx=20). ImageIdx=20 represents a visually difficult condition with darker appearance and high bubble density. Such frames increase sensitivity to contrast, boundary visibility, and post-processing thresholds, and can lead to overprediction (extra instances) or fragmented masks. The qualitative comparison highlights how counting stability can degrade in dense scenes even without strong overlap, reinforcing the need to consider $|\Delta N|$ and ΔN alongside overlap-based metrics.

Case D: small and irregular bubble shapes successfully handled (ImageIdx=3). While error cases are essential, it is also informative to show that the best-performing regime generalizes beyond near-circular bubble shapes. ImageIdx=3 contains small and irregular shapes, yet the transfer configuration remains stable and visually consistent. This case supports the quantitative observation that transfer learning improves generalization in the presence of limited labeled real data, even when bubble morphology departs from idealized shapes.

Summary of qualitative findings. Across the inspected frames, the dominant qualitative error modes are: (i) overcounting driven by spurious instances, (ii) ambiguity introduced by bubble-like artifacts in

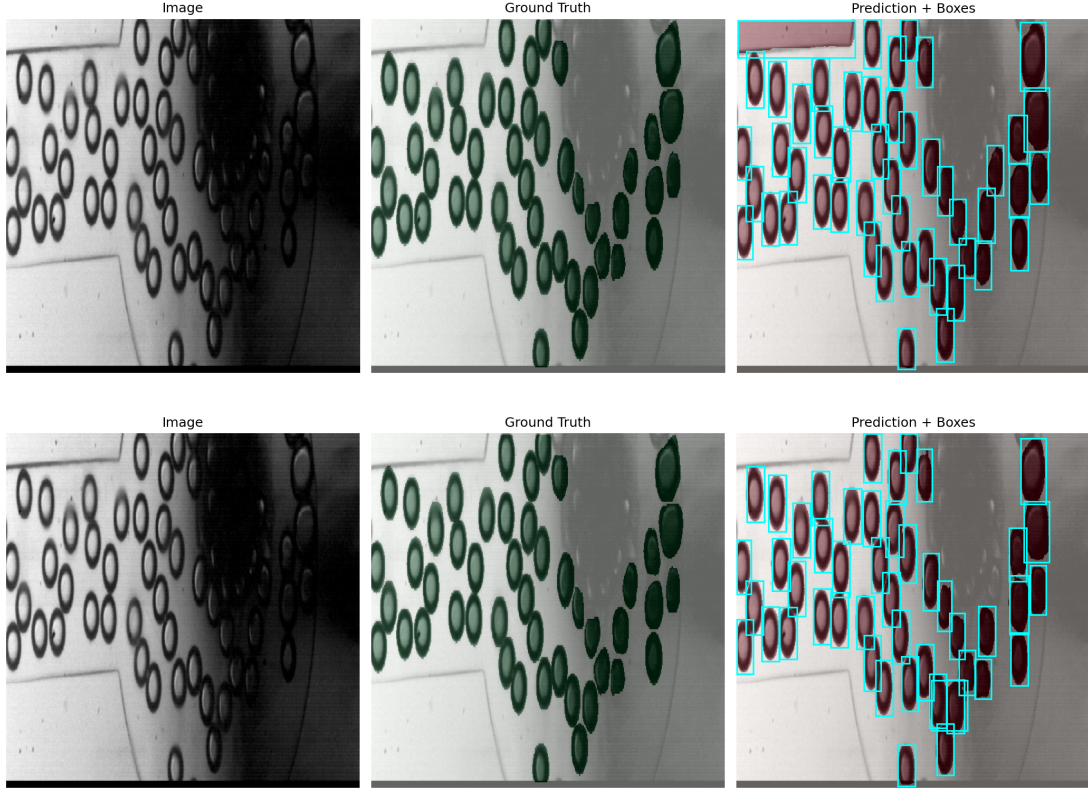


Figure 4.24: Dense, low-visibility case for ImageIdx=20. Comparison between `resnet50_fpn_v2` transfer and `resnet101_fpn` transfer highlights differences in overprediction tendency under darker and denser conditions.

the background, and (iii) increased sensitivity to darker and denser conditions that can promote overprediction. These observations align with the quantitative finding that overlap metrics should be complemented with instance-derived indicators when segmentation is used for downstream measurements.

4.8 Annotation-efficiency results summary

This section reports the results of the annotation-efficiency study described in Section 3.5, designed to quantify how much annotation time can be reduced by using model-assisted workflows. Three annotation modes were compared on the same controlled set of 14 representative experimental images, annotated by a single annotator: (A) fully manual annotation, (B) editing synthetic pre-annotations, and (C) editing synthetic pre-annotations with SAM2-assisted refinement. The objective is to measure the practical time savings achievable when generating a real dataset suitable for fine-tuning and evaluating instance segmentation models on bubbly-flow imagery.

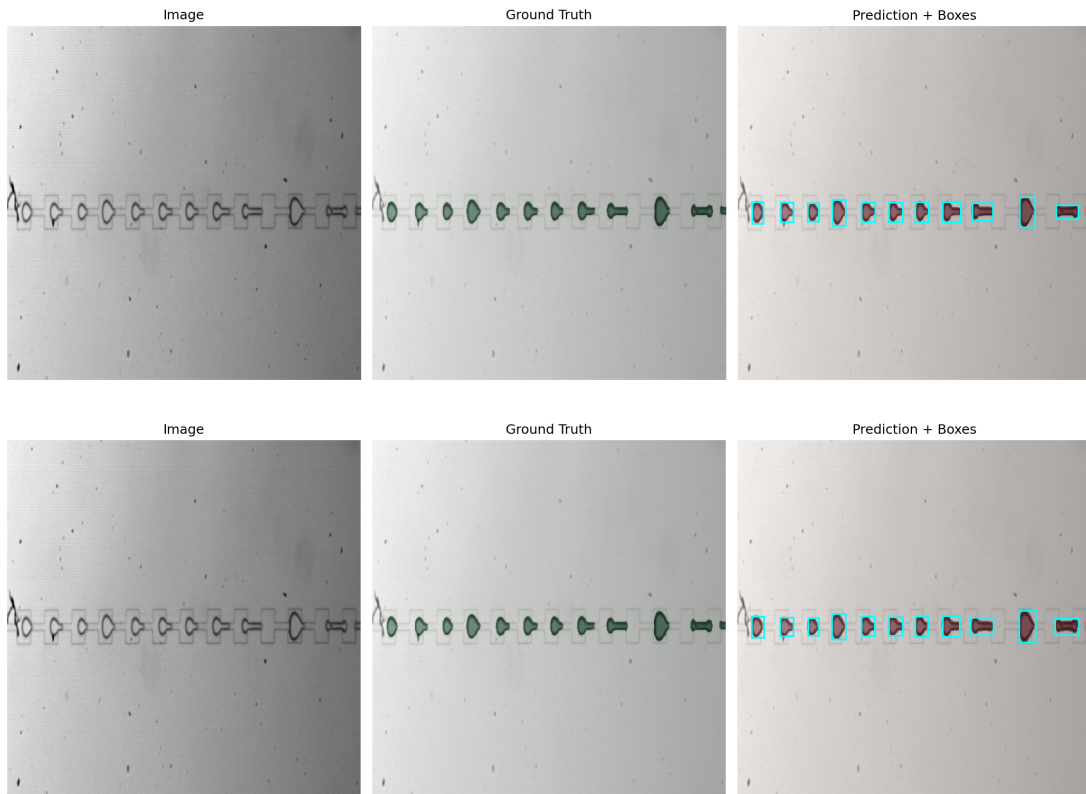


Figure 4.25: Robust success example for ImageIdx=3 with small/irregular bubbles. The transfer regime remains stable, indicating that improved generalization is not limited to near-circular instances.

4.8.1 Quantitative results

4.8.1.1 Time per image

Table 4.8 reports the mean, median, and standard deviation of the annotation time per image for each mode. The results show a strong reduction in annotation time as model assistance increases: the mean time per image decreases from 179.18 s in fully manual annotation (Mode A) to 57.07 s when starting from synthetic pre-annotations (Mode B), and to 25.11 s when combining pre-annotations with SAM2 refinement (Mode C). Relative to the manual baseline, Mode B reduces mean annotation time per image by approximately 68%, and Mode C achieves a total reduction of about 86%.

Table 4.8: Mean, median, and standard deviation of annotation time per image for each mode.

Mode	Mean [s]	Median [s]	Std [s]
Manual annotation (A)	179.18	170.69	88.01
Pre-annotations only (B)	57.07	45.40	35.77
Pre-annotations + SAM2 (C)	25.11	15.34	22.78

Per-image annotation times across the three modes are visualised in Figure 4.26. Each line corresponds to one of the 14 selected images and illustrates both the magnitude of the speedup and the remaining variability across frames.

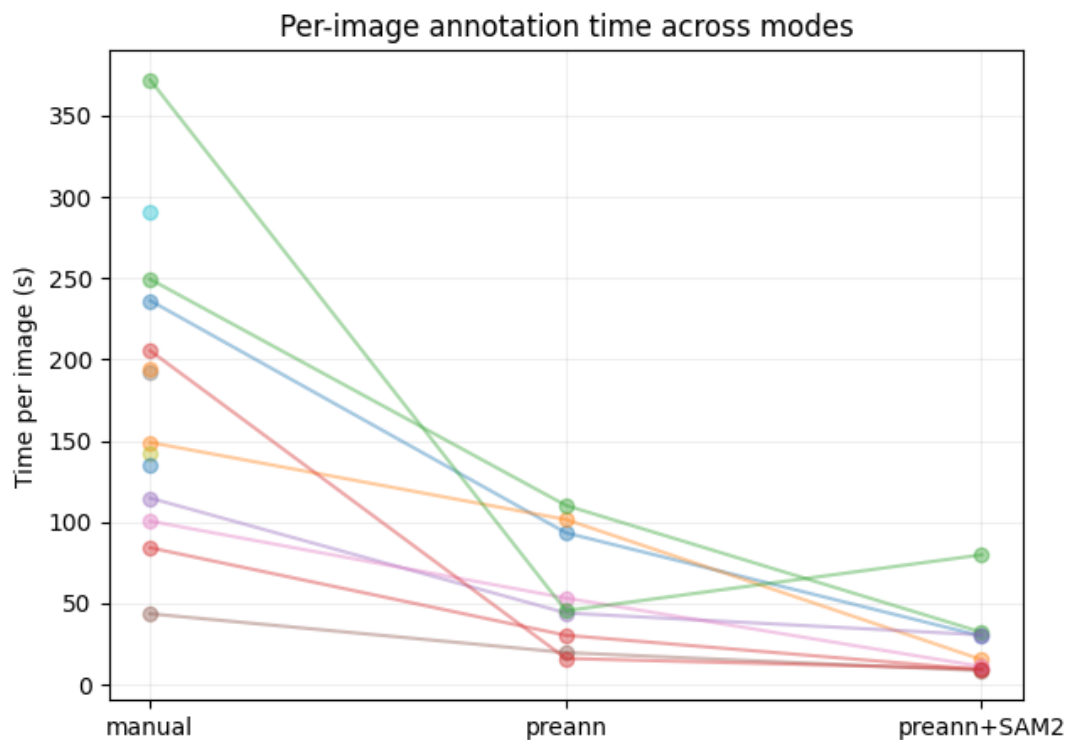


Figure 4.26: Annotation time per image across the three annotation modes. Each coloured line corresponds to one of the 14 selected images.

4.8.1.2 Seconds per bubble

Because the number of bubbles varies across images, annotation effort is also reported as seconds per bubble. This normalisation is important because instance segmentation time tends to scale with the number of instances that must be created, adjusted, or verified. Table 4.9 summarises the seconds-per-bubble statistics for each mode. Fully manual annotation required 94.49 s per bubble on average, whereas synthetic pre-annotations reduced this to 19.43 s per bubble. With SAM2 refinement on top of the pre-annotations, the mean annotation time per bubble decreased to 3.20 s, corresponding to an overall improvement of nearly $30\times$ relative to the manual baseline.

Figure 4.27 visualises the distribution of seconds per bubble per mode. The plot highlights both the large reduction in central tendency and the reduction in variability for the SAM2-assisted workflow.

Table 4.9: Seconds per bubble across different annotation modes.

Mode	Mean [s/bubble]	Median [s/bubble]	Std [s/bubble]
Manual annotation (A)	94.49	96.23	52.90
Pre-annotations only (B)	19.43	10.09	31.51
Pre-annotations + SAM2 (C)	3.20	3.06	2.18

4.8.2 Discussion

4.8.2.1 Effect of synthetic pre-annotations

The comparison between Modes A and B isolates the contribution of synthetic pre-annotations. Even without SAM2, starting from synthetic model predictions reduced the mean time per image by more than 68% and lowered the mean seconds per bubble by approximately a factor of five. This behaviour is consistent with strategies reported in the literature where synthetic images are used to train segmentation models that are then applied to experimental data with limited additional annotation [15, 13].

4.8.2.2 Added value of SAM2 refinement

The difference between Modes B and C quantifies the incremental benefit of SAM2-based refinement. SAM2 reduces the mean per-bubble annotation time from 19.43 s to 3.20 s and the mean time per image from 57.07 s to 25.11 s. These gains are consistent with the reported capabilities of SAM and SAM2 as interactive segmentation models that can produce accurate masks from a small number of prompts [16, 23].

An example of SAM2 producing clean refinements over synthetic pre-annotations is shown in Figure 4.28.

4.8.2.3 Zero-effort cases and variability

For several images, synthetic pre-annotations were already of near ground-truth quality, so no corrections were required in Modes B and C. Label Studio therefore recorded a `lead_time` of zero seconds. These zero-effort cases indicate that, for certain operating conditions, the synthetic-trained model generalised extremely well to real data. At the same time, variability across images was observed: frames containing many small, low-contrast or strongly deformed bubbles tended to require more corrections. In such cases, SAM2 was particularly valuable, as a small number of prompts could rapidly refine boundaries that would otherwise be tedious to correct manually.

4.8.3 Implications and limitations

4.8.3.1 Implications for scaling up annotation

The achieved annotation speeds suggest that larger real datasets can be produced efficiently using this hybrid workflow. Reducing annotation time per bubble from approximately 95 s (manual) to about 3 s

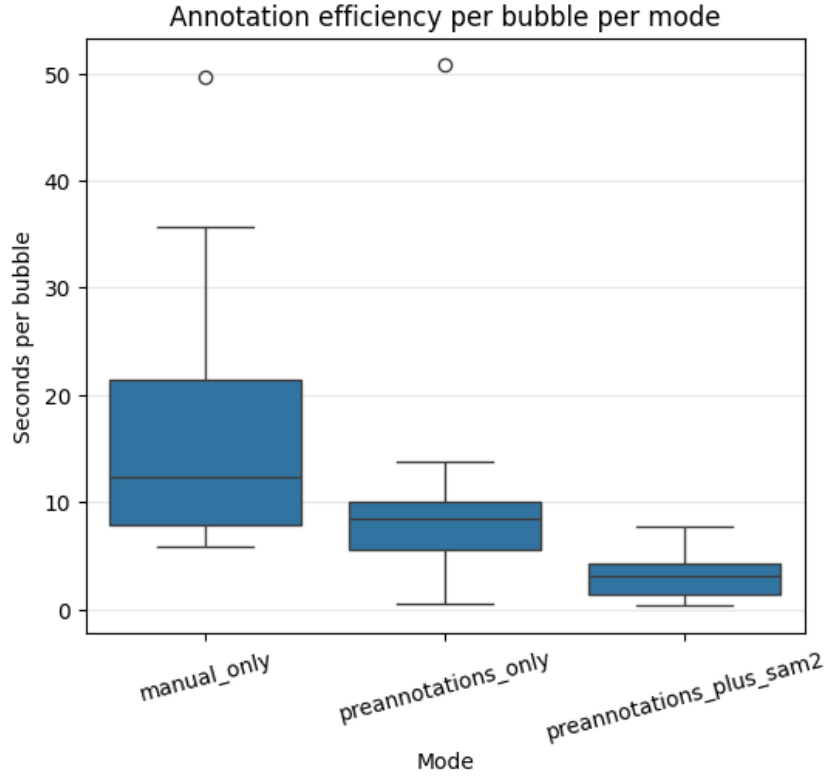


Figure 4.27: Boxplot of seconds per bubble for each annotation mode. The combined synthetic pre-annotation + SAM2 workflow (Mode C) reduces annotation cost by nearly $30\times$ compared with manual annotation.

(synthetic pre-annotations + SAM2) corresponds to nearly an order-of-magnitude increase in effective throughput. Even if these ratios are optimistic when extrapolated beyond the 14 representative images, the gains strongly indicate that several hundred high-quality annotations can be produced within a practical timeframe, enabling robust transfer learning from synthetic to real data [15, 10]. Moreover, the presence of zero-effort cases suggests that, under some conditions, the marginal annotation cost can approach zero. This opens the possibility of future active-learning strategies where annotators focus on the most difficult images, while easy images are accepted directly or only lightly verified.

4.8.3.2 Limitations of the study

Several limitations should be considered when interpreting these results:

- **Single annotator.** All annotations were carried out by a single annotator, which ensures consistency but does not capture inter-annotator variability; time savings may differ for annotators with different levels of expertise.

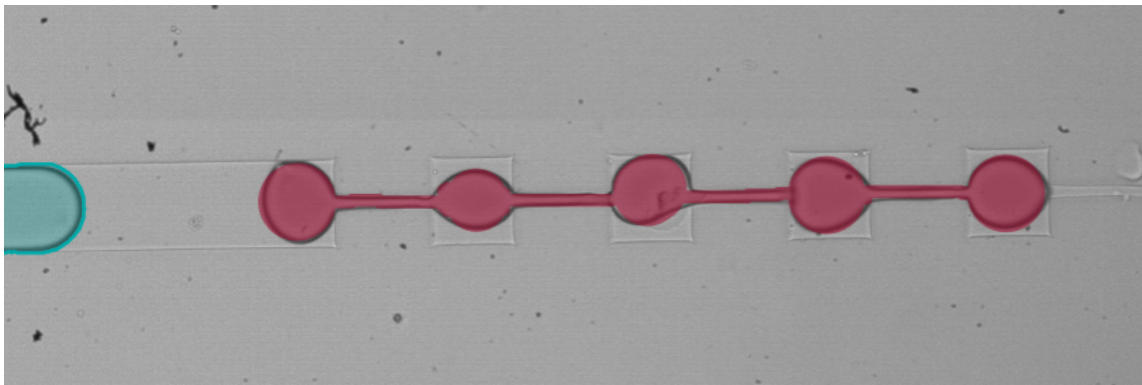


Figure 4.28: Example of SAM2-assisted refinement applied to synthetic pre-annotations, producing accurate bubble masks along the channel.

- **Small sample size.** Only 14 images were used for the controlled comparison. Although the frames were selected to be representative, larger-scale studies would further validate the observed trends.
- **Learning effects.** Because the same 14 images were annotated three times (once per mode), a familiarity effect may be present. This effect likely benefits the later, more automated modes; therefore, the measured speedups are, if anything, conservative with respect to the true benefit of model assistance.
- **Task definition.** The study focused on instance segmentation of bubbles only. Additional tasks such as discarding artefacts, assigning physical attributes, or labeling other flow structures were not timed and could increase annotation complexity in practice.

4.8.4 Summary

The annotation-efficiency study demonstrates that:

- synthetic Mask R-CNN pre-annotations reduced annotation time by approximately 68% per image and by about a factor of five in seconds per bubble compared with fully manual annotation;
- adding SAM2-based interactive refinement on top of these pre-annotations further reduced time by roughly a factor of six, resulting in an overall improvement of nearly $30\times$ in seconds per bubble relative to the manual baseline;
- for several images, synthetic pre-annotations were already of near ground-truth quality, leading to essentially zero-effort annotation in the assisted modes;
- the combined synthetic pre-annotation and SAM2-assisted workflow aligns with modern semi-automatic labeling strategies and foundation-model integration in annotation tools.

These results support the practical feasibility of scaling real annotations to strengthen the synthetic-to-real transfer learning experiments presented in this dissertation.

4.9 Key takeaways

The experimental results lead to the following main conclusions:

- **Transfer learning is the most consistently effective regime.** Initializing from synthetic-trained checkpoints and fine-tuning on real data improves both segmentation overlap and measurement-oriented indicators, with the largest gains observed for the higher-capacity backbone (`resnet101_fpn`). In addition, TensorBoard validation mAP curves show higher validation plateaus under transfer learning, supporting the test-set findings.
- **Overlap metrics and measurement indicators are complementary.** Dice/IoU quantify union-mask agreement but do not fully capture instance split/merge behavior. Reporting $|\Delta N|$ and ΔN is necessary to assess counting stability and systematic over/under-counting, which directly affect downstream bubble statistics.
- **Augmentation during transfer does not improve mean performance in this dataset.** On the 25-image real test split, the Transfer+Aug configuration does not increase mean Dice and generally worsens or leaves unchanged counting stability. ECDF analysis of per-image $|\Delta N|$ indicates that augmentation effects are not uniformly beneficial and may primarily affect the error tail rather than the mean.
- **Backbone choice involves a clear speed–accuracy trade-off.** In the transfer regime, `resnet50_fpn` is fastest, while `resnet101_fpn` is slowest. `resnet50_fpn_v2` provides the best overall compromise, achieving the strongest test-set accuracy with only a moderate reduction in throughput.
- **Difficulty stratification shows where errors concentrate.** Stratifying results by GT bubble density (GT_NumBubbles) and bubble scale (GT_MeanArea) confirms that instance-level errors increase in denser and visually harder frames, even when overlap remains high. These trends reinforce the need for combined evaluation protocols in multiphase-flow measurement contexts.
- **Qualitative inspection confirms the dominant failure modes.** Representative cases show that false positives from bubble-like artefacts and overprediction in dark/dense conditions are key contributors to positive ΔN , while the best-performing configuration remains robust in typical and irregular-shape cases. Backbone capacity does not guarantee improved counting behavior under fixed post-processing thresholds.
- **Annotation efficiency gains make scaling real datasets feasible.** Synthetic pre-annotations reduce annotation time substantially, and SAM2-assisted refinement yields an overall improvement

of nearly $30\times$ in seconds per bubble relative to manual annotation on the controlled 14-image study. This supports the practicality of expanding real training data to strengthen future transfer learning and robustness studies.

References

- [1] *Accelerating Annotation with Model-Assisted Labeling in Label Studio*, 2023.
- [2] *Label Studio Documentation*, 2023.
- [3] *Introducing SAM2: Enhanced Segment Anything Model*, 2024.
- [4] *Using SAM2 for Auto-Annotation in Label Studio*, 2024.
- [5] *TorchVision Documentation: maskrcnn_resnet50_fpn_v2*, 2025. Accessed 2026-01-07.
- [6] Torchvision object detection finetuning tutorial, 2026.
- [7] Transfer learning for computer vision tutorial, 2026.
- [8] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.
- [9] Yizhou Cui, Chengxiang Li, Wanli Zhang, Xiaoqi Ning, Xiaogang Shi, Jinsen Gao, and Xingying Lan. A deep learning-based image processing method for bubble detection, segmentation, and shape reconstruction in high gas holdup sub-millimeter bubbly flows. *Chemical Engineering Journal*, 449, 12 2022.
- [10] Behzad Karkari Gharehchobogh, Ziaddin Daie Kuzekanani, Jafar sobhi, and Abdolhamid Moallemi Khiavi. Flotation froth image segmentation using mask r-cnn. *Minerals Engineering*, 192:107959, 2 2023.
- [11] Tim Haas, Christian Schubert, Moritz Eickhoff, and Herbert Pfeifer. Bubbnn: Bubble detection using faster rcnn and shape regression network. *Chemical Engineering Science*, 216:115467, 4 2020.
- [12] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [13] H. Hessenkemper, S. Starke, Y. Atassi, T. Ziegenhein, and D. Lucas. Bubble identification from images with machine learning methods. *International Journal of Multiphase Flow*, 155, 2022.
- [14] Hendrik Hessenkemper, Lantian Wang, Dirk Lucas, Shiyong Tan, Rui Ni, and Tian Ma. 3d detection and tracking of deformable bubbles in swarms with the aid of deep learning models. *International Journal of Multiphase Flow*, 179:104932, 9 2024.
- [15] Tess A.M. Homan and Niels G. Deen. Deep learning bubble segmentation on a shoestring. *Industrial and Engineering Chemistry Research*, 63(17):7800–7806, 5 2024.

- [16] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [17] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [18] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision – ECCV 2014*, pages 740–755, 2014.
- [19] Ivan Malakhov, Aleksandr Seredkin, Andrey Chernyavskiy, Vladimir Serdyukov, Rustam Mullyadzanov, and Anton Surtaev. Deep learning segmentation to analyze bubble dynamics and heat transfer during boiling at various pressures. *International Journal of Multiphase Flow*, 162:104402, 5 2023.
- [20] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 2010.
- [21] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [22] University Project. Bubann_gan dataset. Roboflow Universe, mar 2024. visited on 2025-11-07.
- [23] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos. *arXiv*, abs/2408.00714, 2024.
- [24] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [25] SciPy Developers. `scipy.stats.ecdf` — empirical cumulative distribution function of a sample. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ecdf.html>. Accessed: 2026-01-20.
- [26] Jee Hyun Seong, Madhumitha Ravichandran, Guanyu Su, Bren Phillips, and Matteo Bucci. Automated bubble analysis of high-speed subcooled flow boiling images using u-net transfer learning and global optical flow. *International Journal of Multiphase Flow*, 159:104336, 2 2023.
- [27] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6:60, 2019.
- [28] Jerol Soibam, Valentin Scheiff, Ioanna Aslanidou, Konstantinos Kyprianidis, and Rebei Bel Fdhila. Application of deep learning for segmentation of bubble dynamics in subcooled boiling. *International Journal of Multiphase Flow*, 169:104589, 12 2023.

- [29] TensorFlow. Transfer learning & fine-tuning, 2026.
- [30] Daizhou Wen, Wuguang Chen, Junlian Yin, Yuchen Song, Mingjun Ren, and Dezhong Wang. Overlapping bubble detection and tracking method based on convolutional neural network and kalman filter. *Chemical Engineering Science*, 263:118059, 2022.
- [31] B. Xin, H. Zhang, and Y. Chen. Semi-automatic annotation strategies in computer vision. *Pattern Recognition*, 113:107804, 2021.
- [32] Haohan Xu, Xin Feng, Yuqi Pu, Xiaoyue Wang, Dingwang Huang, Weipeng Zhang, Xiaoxia Duan, Jie Chen, and Chao Yang. Bubsam: Bubble segmentation and shape reconstruction based on segment anything model of bubbly flow. *AIChE Journal*, 2024.
- [33] Hu Zhang, Zhaohui Tang, Yongfang Xie, Xiaoliang Gao, and Qing Chen. A watershed segmentation algorithm based on an optimal marker for bubble size measurement. *Measurement: Journal of the International Measurement Confederation*, 138:182–193, 5 2019.
- [34] Wen Zhou, Shuichiro Miwa, Ryoma Tsujimura, Thanh Binh Nguyen, Tomio Okawa, and Koji Okamoto. Bubble feature extraction in subcooled flow boiling using ai-based object detection and tracking techniques. *International Journal of Heat and Mass Transfer*, 222:125188, 5 2024.

Index

Multiphase Flow, 1

Segmentation, 1